

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

GRADUATE SCHOOL



MASTER SPACE

M2 INTERNSHIP REPORT

DÉVELOPPEMENT D'UN MODULE D'ORIENTATION DE
NANOSATELLITE POUR LA SUITE LOGICIEL DOCKS POUR LE
PROTOTYPAGE PRÉCOCE DE PROFILS DE MISSIONS SPATIALES

Student:	La Thuy Linh
Academic year:	2022-2023
External Supervisor:	LACREU Feliu
Host Institute:	CENSUS, Observatoire de Paris
Internal Supervisor:	Assoc.Prof. NGO Duc Thanh

Meudon, September 2023

Acknowledgment

Acknowledgment is always the most challenging part to deliver since there are no numbers to count on. I cannot express enough my gratitude to the ones who have contributed to this journey of mine so far. For me, this internship has a rough beginning, a rollercoaster ride marked by ups and downs. But now, in the middle of the journey, I believe that every twist and turn has a meaning.

My deepest appreciation goes to Dr. Boris Segret and Feliu Lacreu. Your mentorship throughout my internship is filled with scientific curiosity, patience, and inspiration. I was lost at the beginning, but with your unwavering supervision and thoroughness, I can make it to the defense.

My pursuit of this master's program cannot be done without the guidance of all the professors, staff, and teachers from USTH and Observatoire de Paris: Prof. Guillaume Patanchon, Prof. Ngo Duc Thanh, Ms. Nguyen Thi Tra, Prof. Daniel Rouan, Dr. Damien Prêle, Dr. Cedric Viou, and Dr. Frédéric Cassaing (ONERA). My gratitude extends to the Vietnam National Space Center for being a launchpad for my academic journey, and for all of the support that I have received from my colleagues here.

I am always immensely grateful for having my parents and my sister by my side. Thank you for all of your care, your faith in me, your unconditional love, and for everything you have done.

I am delighted to have had the opportunity to get to know my colleagues at CENSUS: Elias, Janis, Kirill, and Pedro. Our shared moments - the fish-feeding, chess matches, the aquarium, the Tuesday drink - have created unforgettable memories. I want to thank Atus for his gentleness, thoughtfulness, and dedication, which strengthens me to go through challenges. I extend my thanks to Tam, Giang Tran, Lan Anh, Huong, and all the friends who have remained by my side throughout this journey.

Last but not least, I must express my gratitude to rainbow club, for being suy together. By the time I was typing this acknowledgment, we were group-calling, pushing each other on the final days to the defense. I cannot imagine having this journey in USTH and France without you. Thanks for the adventures, the stories, the sharing, and for letting me be my true self. I wish all the best for us in the future.

Contents

1	Introduction	1
2	The scientific objective of the internship	4
3	Methodology	5
3.1	Quaternions	5
3.1.1	Rotation with arbitrary axis	6
3.1.2	Rotation with fixed axis	7
3.1.3	<i>pyquaternion</i> and cautions	7
3.2	Notations and conventions	8
3.3	Pointing Strategies	9
3.4	Line of Sight	10
3.4.1	Nadir pointing	11
3.4.2	Inertial Pointing	12
3.4.3	Vector to Sun and other bodies in the Solar system	12
3.4.4	Target Pointing	13
3.4.5	Limb tracking	14
3.4.6	Scan	16
3.5	Constraints	16
3.5.1	Sun optimized	16
3.5.2	Rotation with period	18
3.6	Integration of quaternion	19
4	Results	19
4.1	Inputs	19
4.2	Outputs	21
4.3	Case study with Venus Express	22
4.3.1	Phase 1: Observation, Nadir Pointing	25
4.3.2	Phase 2: Observation, Limb tracking	26
4.3.3	Phase 3: Observation, Star Occultation	27

4.3.4	Phase 4: Transmission, Earth Pointing	28
4.3.5	Investigation on the case study	28
5	Discussions	33
6	Conclusions	33
7	Appendix	36
7.1	Quaternion Properties	36
7.2	Reference frames	37
7.2.1	Inertial frame	37
7.2.2	Non-inertial frames	37
7.3	Time scale	39

List of abbreviations

Attitude Determination and Control System	ADCS
Attitude Ephemeris Message	AEM
Barycentric Dynamical Time	TDB
Centre d'Ingénierie Concourante	CIC
Centre National d'Etudes Spatiales	CNES
CEntre pour les Nanosatellites en Sciences de l'UniverS	CENSUS
Consultative Committee for Space Data Systems	CCSDS
Design and Operation Cross-checking Services	DOCKS
Field of View	FoV
Institut de mécanique céleste et de calcul des éphémérides	IMCCE
International Celestial Reference Frame	ICRF
Line of Sight	LoS
Navigation and Ancillary Information Facility	NAIF
Orbit Ephemeris Message	OEM
Spacecraft - Planet - Instrument - C-matrix - Events	SPICE
Visualization Tool for Space data	VTs
YAML Ain't Markup Language	YAML

List of Tables

1	Mission types, requests and constraints	2
2	Conventions	9
3	Case studies for pointing strategies	10
4	Parameters in trajectory file	11
5	Available modes and corresponding parameters for primary payload . .	21
6	Available constraints and corresponding parameters for secondary payload	21
7	VEX's payloads	22
8	Examples of pointing strategies for VEX	23
9	Time scales	39

List of Figures

1	DOCKS Modules, image credit: [7]	3
2	The workflow of DOCK's Quaternion	4
3	Rotation with arbitrary axis	6
4	Rotation with fixed axis	7
5	Satellite's frame and instrument's frame	8
6	Pointing Strategies	9
7	Example of Propagator output file	10
8	Vectors between bodies in ephemeris	12
9	Target pointing configuration	14
10	Limb tracking configuration	15
11	Limb tracking configuration, with a view along the $\vec{PO}$	15
12	Normal view of limb tracking mode	15
13	Scan mode configuration	16
14	Rotation of $\vec{pl}_2$ about $\vec{pl}_1$	17
15	Optima position of $\vec{pl}_2$	18
16	Rotation with fixed period of $\vec{pl}_2$ about $\vec{pl}_1$	19
17	Example of input YAML file	20
18	Example of AEM output file	21
19	Venus - Sun - Earth angle at the beginning of January 2011	23
20	Input YAML file for VEX study case	24
21	Observation mode, Nadir Pointing	25
22	Observation mode, Limb tracking	26
23	Observation mode, Star Occultation	27
24	Earth Pointing mode	28
25	Primary LoS in ICRF (x value: blue, y value : orange, z value: green)	29
26	Sun vector in ICRF (x value: blue, y value : orange, z value: green)	30
27	Angle between primary payload vector and primary LoS	31
28	Angle between secondary payload vector and secondary LoS (Sun vector)	32
29	ICRF and Ecliptic frame, image credit: [27]	37

30	Family of frames, image credit: [21]	38
31	Time scale relations, image adapted from F. Lacreu (CENSUS)	39

Abstract

DOCKS (Design and Operation Cross-checking Services) is a suite of Python-based tools created by CENSUS (Centre pour les Nanosatellites en Sciences de l'Univers), aimed at assisting in the early phases of planning a space mission, particularly for nanosatellites. Within DOCKS, there are existing modules designed to provide ephemeris files for satellite trajectory, intervisibility between the objects, and estimation of required onboard electrical power. Addressing the requirements of satellite's attitude, a new module in DOCKS called "Quaternions", has been developed within this internship. Its main objectives are to ensure the satellite not only fulfills its primary scientific mission but also maintains the health of its components throughout the mission's duration. This objective is achieved by calculating the attitude ephemeris regarding the requirements - or so-called "pointing strategies" - from the mission planner. In this module, numerous common nanosatellites' pointing strategies will be constructed using geometry and quaternion algebra, concerning scientific pointing modes (inertial pointing, nadir pointing, or ground station tracking, etc.) and constraints (avoiding specific exclusion zones, avoiding direct solar flux, etc.). DOCKS' Quaternions is programmed using Python as the programming language, and it provides the satellite's attitude ephemeris data in the form of quaternions. The construction of this module is described in this report, with a case study to demonstrate the module's capabilities.

Keywords: Satellite's attitude, pointing strategies, Quaternions, frame transformation, Python.

1 Introduction

Mission planning and scheduling for satellites has always been a crucial yet sophisticated process in any space mission [1, 2]. The goal of this process is to collect invaluable data within the satellite's time budget while ensuring the status of the spacecraft and its instruments. Yet the complexity arises in the planning process: different types of missions with unique characteristics lead to plenty of specific planning requests, planning cycles, as well as multiple constraints and resources, as shown in Table 1 [1, 2, 3]. On the grounds that the position and velocity of the spacecraft are time-dependent, the time window for observation is limited owing to unavoidable events (occultation, eclipse, or extreme environmental conditions). Meanwhile, while one of the instruments is requested to operate a specific observation mode (target pointing, tracking an object, communicating to a ground station, etc.), the spacecraft's orientation must also ensure the other instrument's status, prevent them from being damaged from the environment (avoid from the over-illuminated condition, avoid heat from the Sun, etc.).

Regarding the mission's complexity, the mission profile ephemeris becomes extremely useful while the planning process takes place. An orbit ephemeris [4] - constructed using the gravitational model and ephemeris of celestial bodies - provides the spacecraft's position, velocity, and acceleration; meanwhile, an attitude ephemeris contains the orientation of the spacecraft [3, 5], and it can be modified frequently. Controlling the satellite's attitude helps the planner to (1) ensure the satellite achieves its requirements, (2) protect the satellite's bus and instruments from being harmed, and (3) predict upcoming available time slots for future observation. However, unlike the orbit ephemeris, the satellite's attitude is not a single-solution problem: multiple attitudes may satisfy the request, or no attitude can meet the criteria of the requested pointing strategy, regarding the position of the satellite, the available time slots, and the constraints.

Considering the needs for the satellite mission planning and mission profile ephemeris, in Centre pour les Nanosatellites en Sciences de l'UniverS (CENSUS [6], Observatoire de Paris), a prototype of a service - Design and Operation Cross-checking Services (DOCKS) [7, 8] - is being built to support the design of satellite mission profile, especially of nanosatellites. Nanosatellites, the main interest of CENSUS, is an economic

Table 1: Mission types, requests and constraints

Type	Examples	Planning requests and cycles	Resources and Constraints
Earth Observations	ESA's Copernicus Sentinel, EUMETSAT's MSG	Planning is separated for payload and satellite, depends on orbit type. Either time of observation or observation is considered.	Compatibility between operation and resources.
Fly-by	Cassini (NASA/ESA/ASI), Rosetta (ESA/NASA/DLR/CNES)	Since fly-bys have a special trajectory, the geometries and distances change rapidly with time. Observation cycles are short-duration with limited operation window. Hence the request of geometry is often a crucial part (e.g. view a target with specific illumination, viewing angle, distance, etc.)	Due to the swift change in geometries and distances, spacecraft pointing is closely related to the observation time with specific geometries, which is also considered to be a resource.
Observatory	James Webb Space Telescope (NASA), Solar Orbiter (ESA)	A planning request is a combination of pointing strategies with a fixed time/ target or open time/ target in the celestial sphere, meanwhile the environmental conditions and constraints are satisfied. Composed of cycles: (e.g. routine calibration plan), long-term (months to year), medium-term (one or several weeks), short-term (one to several days), Target of Opportunity (quick planning for observation of an ideal astronomical object)	Different types of resource and constraint are under consideration: – Self or Inter-instrument Constraints (e.g. mechanical or electrical performance) – S/C Resource Utilization (e.g. power, link budget, memory) – Pointing Constraints: typically, Sun and other object illumination avoidance, occultation, etc.
Planetary	JUICE (ESA), BepiColombo (ESA/JAXA)	Contains the information about the target (global constraints, geometric constraint, referenced instruments that will be utilized, etc.), and scientific justification for the request. The planning process of planetary missions typically follows a three-phase planning approach with long-term (months), medium-term (weeks - months), and short-term planning(days - weeks).	Resources: – Spacecraft Attitude for specific pointing – Power and Data Storage Capacity Constraints: – Geometric Constraints: Illumination conditions (solar local elevation, phase angle, distance, etc.); – Thermal Constraints: Maximum illumination period of certain areas of the spacecraft and its instruments

sector of satellites with a mass smaller than 50 kilograms. They have had a potent development over the last two decades: from serving the purpose as student projects, they now can perform education mission, technology mission, scientific mission, and be the test platform for Technology Readiness Level (TRL). Promptly and robustly advanced, nanosatellites now have gone beyond Low Earth Orbit (LEO) and can perform missions in Geostationary Orbit, Lunar, Mars flyby, and other interplanetary missions [9, 10]. To plan altogether for a nanosatellite mission, DOCKS approaches all aspects of a satellite mission with various computational modules: Deep Space Propagator [11], Intervisibility, Attitude Determination and Control System (ADCS), Electric Power System (EPS), CONIC, Datalink, and Quaternions, as shown in Figure 1. Up to now, Propagator and Intervisibility modules have been developed and validated.

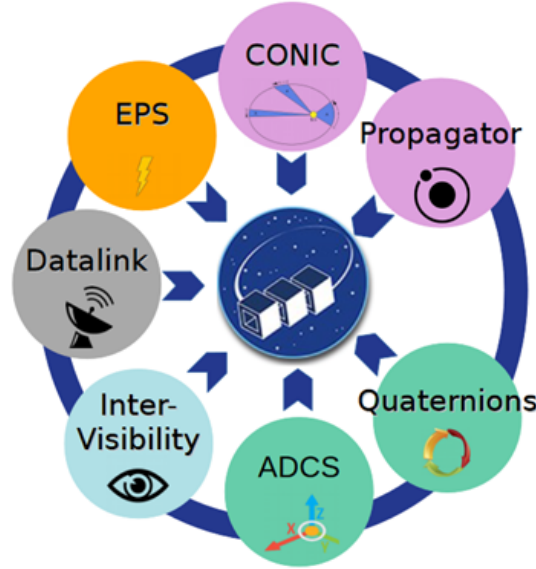


Figure 1: DOCKS Modules, image credit: [7]

Nanosatellite's pointing strategies may be simpler than those of large space missions, but they are derived from the same purposes and constraints stated in Table 1. In the project timeline, pointing strategies are studied very early, and the attitude of a nanosatellite is a problem arising very quickly when the discussion takes place between the scientific team and the technical team. On the grounds of solving this problem, DOCKS's Quaternions/ Pointing strategies is expected to create quaternion-based attitude ephemeris for modeling specific pointing strategies. Within DOCKS, the Quaternions module can take the orbit ephemeris from the Propagator module and event file

from the Intervisibility module as inputs and will produce the output which can serve as inputs for the ADCS module. Outside DOCKS, users can use the attitude ephemeris independently to plan the schedule for the mission. Quaternions are chosen to be the attitude representation in this module, after careful consideration of various representations: quaternion, Euler Angles, Rodrigues parameters, Direction Cosine Matrix, Spin axis [5, 12]. Each among those representations has its advantages and limitations, however, the use of quaternions is the most computational-friendly since it has the least number of parameters (four parameters) and no singularity. It has diverse applications in mathematics, physics, computer graphics, and computer vision because quaternion algebra allows efficient rotation computations. However, the rotation by a quaternion is not easily conceivable for the user since it uses four parameters to present orientation in 3-D space. Confusions are also frequently raised between the quaternion users about the rotation coordinate and vector rotation [13, 14], hence, the quaternion user has to be circumspect with the perplexings that may arise.

2 The scientific objective of the internship

The objective of this internship is to configure the attitude of the satellite for different pointing strategies and express this optimized attitude using quaternion. It requires an understanding of pointing strategies, frames, and transformation of vectors and frames. The functions that are used to provide the attitude ephemeris will be coded in Python and integrated into the current DOCKS' environment along with other modules.

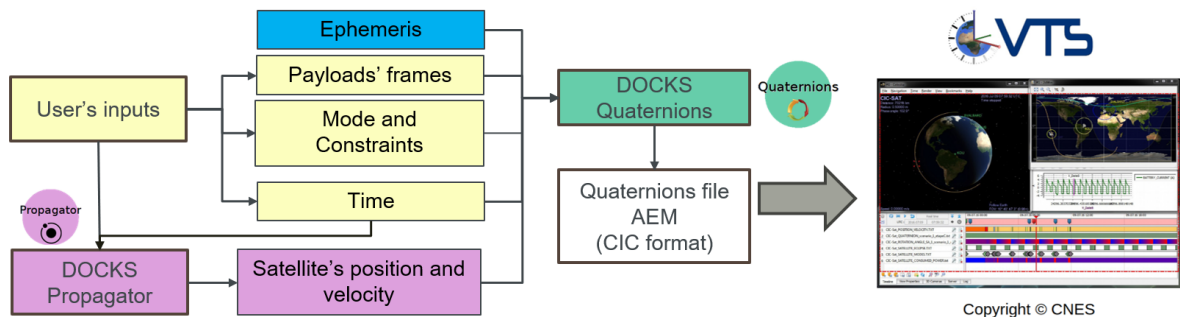


Figure 2: The workflow of DOCK's Quaternion

Figure 2 shows the general flow of the use of DOCKS' Quaternions. The inputs of

the user are indicated in the yellow blocks, which are: the payloads' frames follow the conventions in section 3.2; the requested modes and constraints, described in section 3.3; and the time of the simulation. The purple block, indicating the input file from DOCKS' Propagator, contains information about the satellite's trajectory, and will be defined in section 3.4. The information of the bodies in the Solar System (blue block) is extracted from ephemeris data, depicted in section 3.4.3. All these inputs will be loaded to DOCKS' Quaternions in a YAML (YAML Ain't Markup Language) file and the outcome will be stored in an Attitude Ephemeris Message (AEM), described respectively in section 4.1 and 4.2. This AEM file, in CIC (Centre d'Ingénierie Concourante) format, can be visualized by VTS (Visualization Tool for Space Data, CNES) [15].

3 Methodology

In this section, with geometry and quaternion algebra, the pointing strategies and optimization algorithm will be explained.

3.1 Quaternions

Quaternion q presenting a rotation about an axis $\vec{v}$ with a certain angle θ is written as:

$$q \equiv (s, \vec{v}) \equiv (\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2}) \cdot \vec{v}) \quad (1)$$

$\vec{v}$ is the vector indicating the rotational axis, presented in 3D space as $\vec{v} \equiv (v_x, v_y, v_z)$ ($\vec{v}$ is normalized), and hence, we can write quaternion q as:

$$q \equiv (\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2}) \cdot (v_x, v_y, v_z)) \equiv (q_0, q_1, q_2, q_3) \quad (2)$$

with $\cos(\frac{\theta}{2})$, $\sin(\frac{\theta}{2}) \cdot v_x$, $\sin(\frac{\theta}{2}) \cdot v_y$, and $\sin(\frac{\theta}{2}) \cdot v_z$ respectively are q_0 , q_1 , q_2 , and q_3 . Other formulas regards the quaternion computation can be found in Appendix 7.1.

A noteworthy characteristic of a quaternion is that $q = -q$, which accords: a rotation about the opposite direction by a negative angle will have the same effect as the original rotation. Despite the two rotations leading to one result, the q with the negative scalar part presents the “longer” rotation [16, 17], thus, the one with the positive scalar part is preferred. Thanks to this characteristic, a quaternion representation is easily represented in scalar-positive format. This is also the format of the quaternion demonstrated in the AEM output file, to assure the unique presentation of a rotation. The automatic change

of sign is performed by *pyquaternion* - the Python package that will be mentioned in section 3.1.3 [17], and can be observed in the results in section 4.

A vector in 3-D can be written in the form of 4-parameter: $\vec{v} \equiv (0, \vec{v})$. A rotation of this vector by quaternion q can be performed by:

$$\vec{v}' = q \otimes \vec{v} \otimes q^* \quad (3)$$

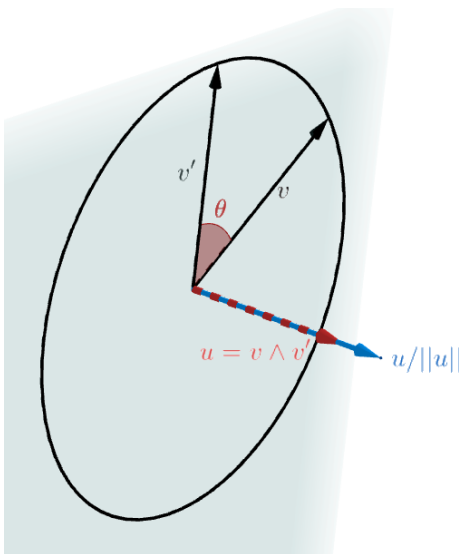
with q^* the conjugate of quaternion q , and $\vec{v}'$ is the vector $\vec{v}$ after the rotation. It is straightforward to build a quaternion if the rotational axis and the angle are known: a quaternion is built by the definition in Equation 2.

Nevertheless, in most of the cases, $\vec{v}$ the starting vector and $\vec{v}'$ the desired position is determined, and a quaternion to transform between these two vectors needs to be constructed without the knowledge of the rotational axis or the rotational angle. With given $\vec{v}$ and $\vec{v}'$, the operation for pointing strategies usually deals with two cases:

1. Rotation with arbitrary axis, or
2. Rotation about a given axis. This case takes place when it is required to maintain the direction of a vector in a certain direction. As a result, this vector will be taken as a rotation axis to create a quaternion.

3.1.1 Rotation with arbitrary axis

A quaternion representing a rotation with arbitrary axis is produced as follow:



Rotational axis:

$$\vec{u} = \vec{v} \wedge \vec{v}'$$

Rotation angle:

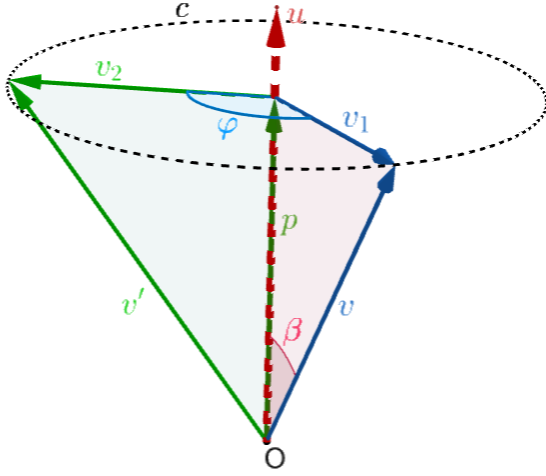
$$\theta = \arccos(\vec{v} \cdot \vec{v}')$$

Quaternion:

$$q = \left(\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right) \frac{\vec{u}}{||\vec{u}||} \right)$$

Figure 3: Rotation with arbitrary axis

3.1.2 Rotation with fixed axis



Assume it is required to rotate the vector $\vec{v}$ around the known rotational axis $\vec{u}$ ($\vec{u}$ is normalized). Let β the angle between $\vec{u}$ and $\vec{v}$, hence β is also the angle between $\vec{u}$ and $\vec{v}'$, and:

$$\beta = \arccos(\vec{u} \cdot \vec{v})$$

Let $\vec{p}$ the projection of $\vec{v}$ on $\vec{u}$, thus:

$$\vec{p} = \cos(\beta)\vec{u}$$

Figure 4: Rotation with fixed axis

$\vec{v}_1$ and $\vec{v}_2$ subsequently the projection of $\vec{v}$ and $\vec{v}'$ on to the plane perpendicular to $\vec{u}$ like in Figure 4 (the circle c lies in that plane).

$$\vec{v}_1 = \vec{v} - \vec{p}$$

$$\vec{v}_2 = \vec{v}' - \vec{p}$$

The angle φ between $\vec{v}_1$ and $\vec{v}_2$ will then be the angle of rotation, and

$$\varphi = \arccos(\vec{v}_1 \cdot \vec{v}_2)$$

The quaternion q that satisfies the demand is:

$$q = (\cos(\frac{\varphi}{2}), \sin(\frac{\varphi}{2})\vec{u})$$

3.1.3 *pyquaternion* and cautions

Three available Python-based packages are built for quaternion: *pyquaternion* [17], *numpy quaternion* [18], and *scipy_rotation* [19]. While *pyquaternion* and *numpy quaternion* are “scalar first” convention, *scipy_rotation* is “scalar last” convention. Furthermore, *pyquaternion* allows access to each element of the quaternion (can access the scalar part and vector part separately). Thus, *pyquaternion* package is the most favorable.

About *pyquaternion*, some unclear information that the document of the *pyquaternion* does not mention, and the user has to make assumptions during the process of building

a quaternion using *pyquaternion*:

1. The function to generate a quaternion in *pyquaternion* is: $q = \text{Quaternion}(\text{axis} = \vec{u}, \text{angle} = \theta)$, with $\vec{u}$ is the axis of rotation, and θ is the angle of rotation, ranges from 0 to 2π . However, it is required that the norm of q to be 1, and norm of $\vec{u}$ to be 1. If and only if then, the conjugate of q and the inverse of q is identical. While, the process of finding the conjugate of q is very easy, finding the inverse of q is not obvious if the norm of quaternion is not equal to 1. A quaternion with a non-unit vector is hard to conceive, hence, it is crucial to make sure that the rotation axis is a unit vector and norm of the quaternion equals 1. *pyquaternion* automatically unitize the rotational axis, but it is not stated so in its documentation, so an additional step of unitize the rotational axis is performed when the function *Quaternion()* is called.

2. The rotational angle θ sometimes confuses, because the angle that needed to be put in *Quaternion()* function is the true angle between 2 vectors, meanwhile the quaternion mathematics concerned with the value $\theta/2$ (Equation 2).

3.2 Notations and conventions

The quaternion expression throughout this document is the scalar-first convention, which means the scalar part will be the first element of the quaternion representation. The quaternion expression is also scalar-positive, as reasoned in section 3.1.

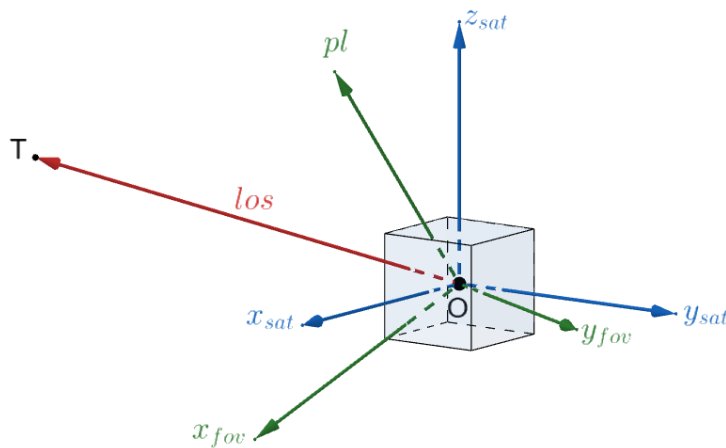


Figure 5: Satellite's frame and instrument's frame

The Figure 5 represents the satellite frame $(O, x_{sat}, y_{sat}, z_{sat})$ colored blue, the FoV frame $(O, x_{fov}, y_{fov}, pl)$ colored green, and the Line of Sight $\vec{los}$ colored red. The vectors

and frame are defined in Table 2.

Table 2: Conventions

	Notation	Implication	Note
International Celestial Reference Frame	ICRF	The inertial frame, based on the Earth Equator and the equinox	More explanation in Appendix 7.2
Satellite frame		The frame associated to the satellite’s bus	
Field of View frame	FoV, fov	The frame associated to the payload	
Line of Sight	LoS, $\vec{los}$	The desired pointing direction	Mostly presented in ICRF
Payload vector	$\vec{pl}$	The boresight of the payload.	Presented in the satellite frame

In this stage of the module, the instrument is considered to be fixed to the satellite, i.e. the payload vector $\vec{pl}$ is unchanged with time. For convenience, throughout this document, the payload vector of any instrument is identical to its $+z$ vector in its Field of View frame, albeit it is not compulsory in real-life situations. A family of non-inertial frames is depicted in Appendix 7.2, and specifically in [20, 21]. The celestial body that the satellite orbits or fly-by is called the “body”, for instance, for Earth observation satellite, Earth is called the “body”. The words “payload” and “instrument” have equal meaning in the context of this report.

3.3 Pointing Strategies

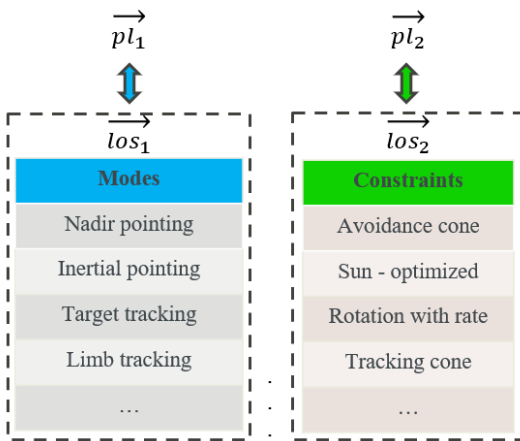


Figure 6: Pointing Strategies

Figure 6 shows the possible “modes” and “constraints” that are usually contemplated in a satellite mission. The strategies being developed only involve strictly two payloads, since one payload would make the pointing strategies underconstrained and three payloads cause overconstrained. The primary payload, which takes the payload vector $\vec{pl}_1$ as boresight, will be assigned to a “mode”, and will be aligned to the corresponding $\vec{los}_1$.

The secondary payload with boresight $\vec{pl}_2$, meanwhile, is assigned to a “constraint”

corresponding to the LoS $\overrightarrow{los_2}$. Unlike the primary payload, the proper position of $\overrightarrow{pl_2}$ not only concerns the relative position of the two payload vectors but also takes into account the constraint with the $\overrightarrow{los_2}$. Several CubeSat missions are examined as case studies for having various pointing strategies and different purposes, outlined in Table 3.

Project	Primary Payload's modes	Constraints
BIRDY	Inertial pointing	Sun-optimized on $\overrightarrow{pl_2}$
COSMOSCal	Tracking target	Rotation around $\overrightarrow{pl_1}$ with period P
METEORIX	Nadir pointing	Sun-optimized on secondary payload
SCION	Limb observation	Sun-optimized on $\overrightarrow{pl_2}$
TERACUBE	Inertial pointing	Rotation around $\overrightarrow{pl_1}$ with period P

Table 3: Case studies for pointing strategies

3.4 Line of Sight

DOCKS's Propagator module produces the information on the trajectory of the satellite in a text file in CIC format. Figure 7 is an example of a trajectory file generated by DOCKS' Propagator.

```

CIC_OEM_VERSION = 2.0
CREATION_DATE   = 2023-08-10T14:30:41
ORIGINATOR      = DOCKS

META_START

COMMENT          DOCKS Propagator version 6.1.0
COMMENT          Propagator method : RK4
COMMENT          Column 1:2 Unit : Time (MJD)
COMMENT          Column 3:5 Unit : Position (KM)
COMMENT          Column 6:8 Unit : Velocity (KM/S)
COMMENT          Column 9:11 Unit : Acceleration (KM/S^2)

OBJECT_NAME      = DOCKS_Sat
OBJECT_ID        = CORPS

CENTER_NAME      = VENUS
REF_FRAME        = ICRF
TIME_SYSTEM      = TDB

META_STOP

59215    0.000    6552.0000    0.0000    0.0000    0.0000    0.0000    7.3870    0.0000    0.0000    0.0000
59215    10.000   6551.6216    0.0000    73.8684   -0.0757    0.0000    7.3866   -0.0076    0.0000   -0.0001
59215    20.000   6550.4866    0.0000   147.7282   -0.1513    0.0000    7.3853   -0.0076    0.0000   -0.0002
59215    30.000   6548.5951    0.0000   221.5710   -0.2270    0.0000    7.3831   -0.0076    0.0000   -0.0003
59215    40.000   6545.9473    0.0000   295.3882   -0.3026    0.0000    7.3802   -0.0076    0.0000   -0.0003
59215    50.000   6542.5437    0.0000   369.1714   -0.3781    0.0000    7.3763   -0.0076    0.0000   -0.0004
59215    60.000   6538.3848    0.0000   42.9118   -0.4536    0.0000    7.3716   -0.0075    0.0000   -0.0005

```

Figure 7: Example of Propagator output file

A clearer explanation of the information in Figure 7 is provided in Table 4.

Table 4: Parameters in trajectory file

Columns	Meaning	Unit	Notations
1	Date in MJD	Day	
2	Seconds elapsed since the beginning of the corresponding date	second	
3, 4, 5	Position in ICRF	km	p_{sat}
6, 7, 8	Velocity in ICRF	km/s	v_{sat}
9, 10, 11	Acceleration in ICRF	km/s ²	a_{sat}

The values in the two first columns are usually treated to generate single value representing time. Let's call it the *time* value in Modified Julian Date (MJD), denoted t , and given by:

$$t = \text{value in first column} + \text{value in second column}/86400$$

with 86400 the number of seconds within a day. Hence, from this trajectory file, it is possible to extract $\vec{p}_{sat}$ the satellite's position vector, $\vec{v}_{sat}$ the satellite's velocity vector at every time step. These vectors will then be exploited to generate the Line of Sight at the corresponding time.

In the Figures 8, 9, 10, 11, 12, and 13 there are some shared notations:

- T: The target that the $\vec{los}$ points to. The length of OT equals the sum of the altitude and the radius of the body.
- P: The satellite
- O: center of the body

Hence, vector $\vec{OP}$ is identical to $\vec{p}_{sat}$.

3.4.1 Nadir pointing

For the satellite to point to the Earth's center, the satellite's position vector is in the opposite direction to the Line of Sight of the payload. In Figure 9, the nadir vector $\vec{nd}$ is opposite in direction with $\vec{p}_{sat}$. Hence,

$$\vec{nd} = -\vec{p}_{sat} \quad (4)$$

3.4.2 Inertial Pointing

Inertial pointing is pointing to a fixed object in the inertial frame, for instance, stars and galaxies. The positions of these objects are expressed in terms of Right Ascension (RA) and Declination (Dec) in spherical coordinates. The RA - Dec value of a celestial object can be transformed to the Cartesian frame by using the following formula:

$$\vec{e} = \begin{bmatrix} \cos(RA)\cos(Dec) \\ \sin(RA)\cos(Dec) \\ \sin(Dec) \end{bmatrix} \quad (5)$$

Thus $\vec{e}$ is the Line of Sight for inertial pointing.

3.4.3 Vector to Sun and other bodies in the Solar system

The Navigation and Ancillary Information Facility (NAIF), NASA provides an information system called SPICE (Spacecraft - Planet - Instrument - C-matrix - Events), along with a Python package named *spiceypy* which is a Python version of the SPICE toolkit. SPICE contains rich ephemeris data of the Solar System bodies and also the large space missions. Inside SPICE, different types of ephemeris data are classified into multiple “kernels”. To extract the Solar System bodies’ trajectory ephemeris, the Spacecraft and Planets Kernels (spk) [22] are furnished and read by *spiceypy* module.

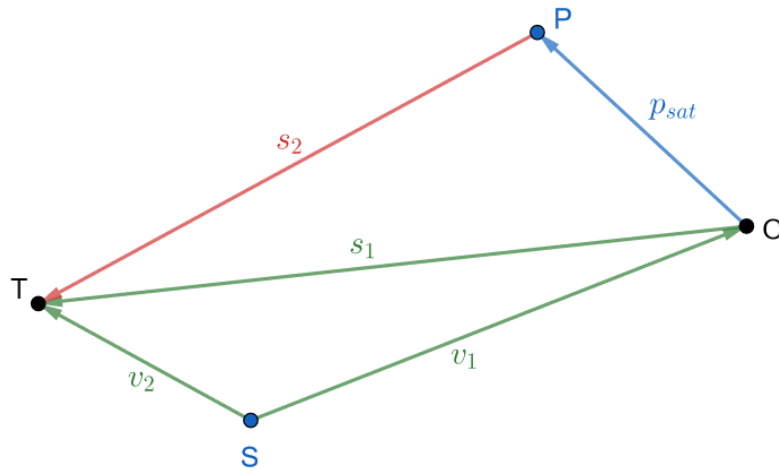


Figure 8: Vectors between bodies in ephemeris

In Figure 8, S is the Solar System Barycenter, O is the center of the body, P is the position of the satellite with respect to the body, and T is another body's center in the Solar System.

The ephemeris file contains the coordinate of points T and O in the ICRF in a given time t , i.e. the ephemeris contains the value of $\vec{v}_1$ and $\vec{v}_2$. The relative position of point T to point O , i.e. the vector $\vec{s}_1$ can be calculated by:

$$\vec{s}_1 = \vec{v}_2 - \vec{v}_1$$

The value of $\vec{s}_1$ can be obtained automatically using the *spiceypy* module. Nonetheless, since the satellite distances from point O by vector $\overrightarrow{p_{sat}}$, the vector from the satellite to point T needed to be obtained by:

$$\vec{s}_2 = \vec{s}_1 - \overrightarrow{p_{sat}}$$

The $\vec{s}_2$ vector will play the role of the Line of sight vector $\overrightarrow{los}$. If point T is the center of the Sun, the Sun vector is obtained: $\vec{S} \equiv \vec{s}_2$.

An online ephemeris model is also developed by in Institut de mécanique céleste et de calcul des éphémérides (IMCCE), Observatoire de Paris [23]. Since this ephemeris does not have a Python compatible module, the SPICE is more favourable for the building of DOCKS' module.

3.4.4 Target Pointing

Since the target point has constant latitude and longitude with respect to the body's surface, it rotates with the body and is fixed to a "body-fixed" frame. To have this position in the ICRF, information about the transformation of this body-fixed frame to ICRF is required.

Given a target that is fixed to this body-fixed frame, with latitude ϕ , longitude L , and altitude alt from the surface. The coordinate of this target in a body-fixed frame is called $\vec{t}_{bf}$. The SPICE provides specific kernels that contain the information about this body-fixed frame of the Solar System's planets and moons, called Planetary Constants Kernel (pck) [24]. Using *spiceypy*, the transformation matrix $[R_{bf2i}]$ between the body-fixed frame to ICRF can be extracted. To have the coordinate of the target in the ICRF called $\vec{t}_i$, a multiplication between R_{bf2i} and $\vec{t}_{bf}$ is performed:

$$\vec{t}_{bf} = \begin{bmatrix} \cos(L)\cos(\varphi) \\ \sin(L)\cos(\varphi) \\ \sin(\varphi) \end{bmatrix} \Rightarrow \vec{t}_i = [R_{bf2i}] \cdot \vec{t}_{bf} \quad (6)$$

In Figure 9, the position of this target is T , with altitude alt from the ground, and the body's radius is R_b . $\vec{OT}$ is the position vector of this point, in ICRF, thus $\vec{OT} = (R_b + alt) \vec{t}_i$. To calculate the LoS $\vec{los}$ in Figure 9, a vector subtraction is performed:

$$\vec{los} = \vec{OT} - \vec{p}_{sat} \quad (7)$$

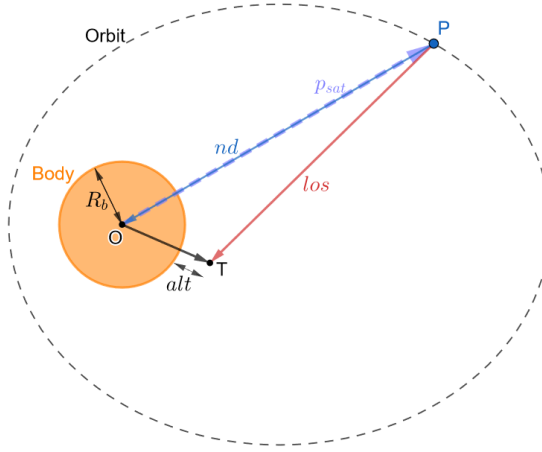


Figure 9: Target pointing configuration

3.4.5 Limb tracking

In limb tracking mode, the satellite can look at different heights of the atmosphere. In Figure 10, a imaginary sphere is built (the green sphere), whose center is the body's center O and its radius equals the sum of the body's radius and a certain desired altitude from the ground. The target point T will lie on this sphere, and by definition of limb tracking, the LoS $\vec{PT}$ is tangent to this sphere, i.e. $\vec{PT}$ is perpendicular to $\vec{OT}$.

It is noticeable that there is a set of points whose connection line to the satellite's position P is tangent to the sphere. This set of points forms a circle, which can also be described as a base circle of a right circular cone, which takes the nadir vector as height (the orange circle in the figure, with the center N) and P as the apex. To specify the target point T in this circle, a referenced point Q is defined. Q lies on this circle, and at the same time, it lies on the orbital plane, at the same side with velocity vector $\vec{v}$ to

the nadir vector $\vec{OP}$. Since the velocity vector $\vec{v}$ also lies on the orbital plane, it can be used to define vector $\vec{PQ}$. To find the $\vec{los}$, the vector $\vec{PQ}$ is built, and then rotated around the nadir vector $\vec{PO}$ by an angle κ . On the circle (N, NQ) , if Q and $\vec{NQ}$ are respectively reference point and vector, then κ is the offset angle of vector $\vec{NT}$ to vector $\vec{NQ}$, as shown in Figure 10 and 11. Each value of κ corresponds to a position of point T on this circle, hence point T and $\vec{PT}$ can be defined via κ . The user can choose the value of κ from 0 to 360° .

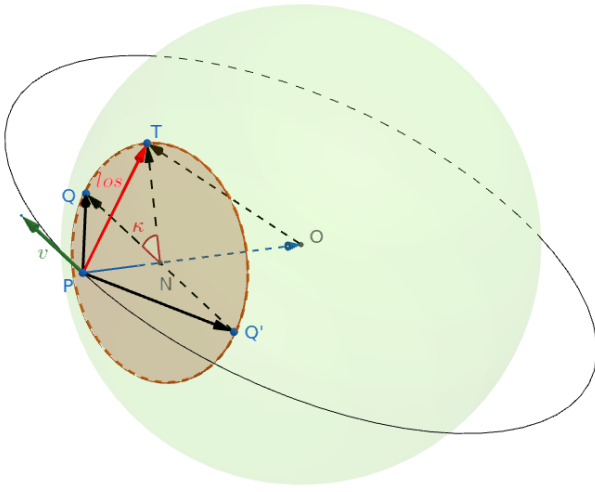


Figure 10: Limb tracking configuration

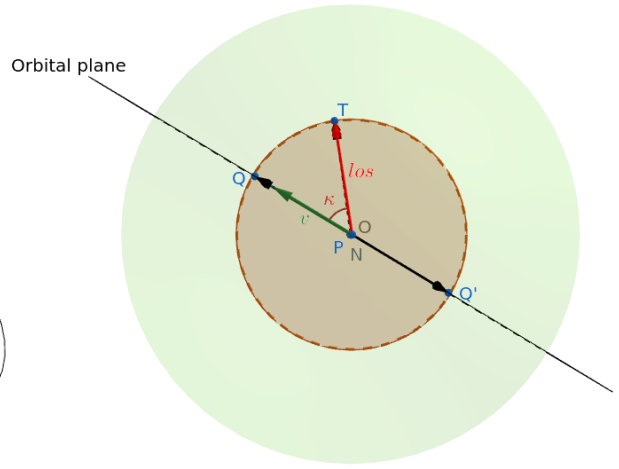


Figure 11: Limb tracking configuration, with a view along the $\vec{PO}$

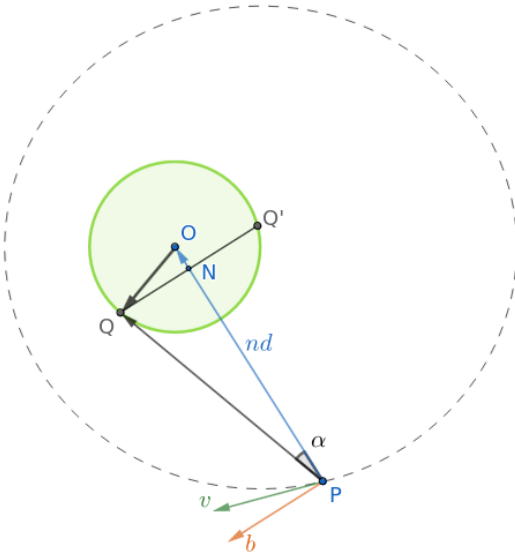


Figure 12: Normal view of limb tracking mode

Now quaternion can be created to rotate vector $\vec{PQ}$ by the angle κ by the axis $\vec{PO}$,

To find the vector $\vec{PQ}$, follow the conventions in Figure 12, the procedure is:

$$\alpha = \sin^{-1}(\|OT\| / \|\vec{nd}\|)$$

$$\vec{b} = (\vec{nd} \wedge \vec{v}) \wedge \vec{nd}$$

$$\vec{PQ} = \cos(\alpha)\vec{nd} + \sin(\alpha)\vec{b}$$

using equation 2:

$$q = \left(\cos\left(\frac{\kappa}{2}\right), \sin\left(\frac{\kappa}{2}\right) \frac{\vec{PO}}{\|\vec{PO}\|} \right)$$

The LoS will then be:

$$\vec{los} = q \otimes \vec{PQ} \otimes q^*$$

3.4.6 Scan

The scan mode configuration is quite similar to the limb tracking mode, but instead of defining the angle between the nadir vector $\vec{nd}$ and the Line of Sight $\vec{los}$, the user can define the angle γ between vector $\vec{PQ}$ and the velocity vector as in Figure 13 (Q remains lying on the surface of the sphere and the orbital plane). Since the angle γ is defined concerning the velocity vector, the use of this mode given a highly elliptical orbit trajectory is not recommended, because the velocity vector changes vastly in one period, with respect to the body. The angle κ is defined as in the limb tracking case. The construction of the Line of Sight $\vec{los}$ is done as follow:

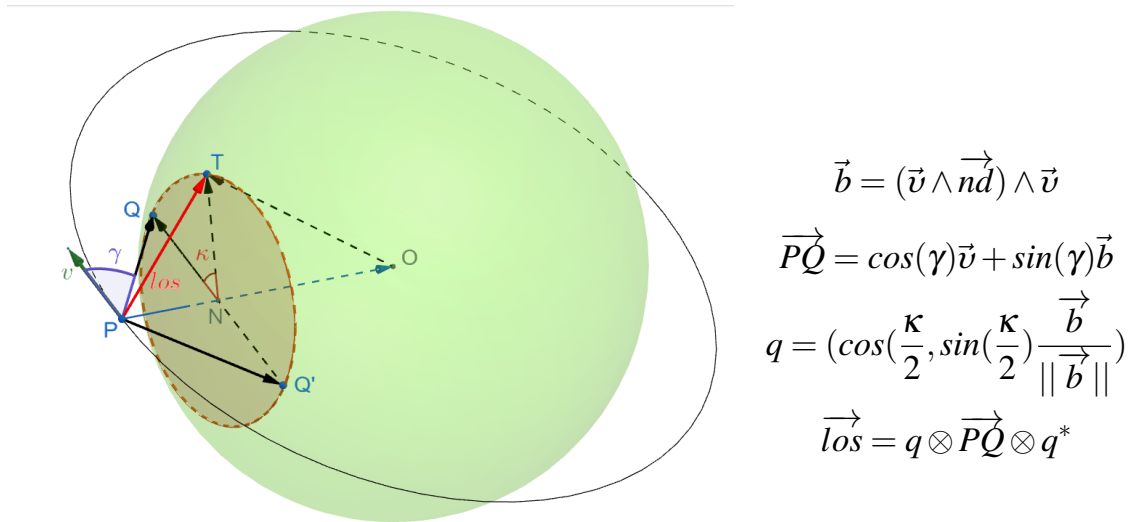


Figure 13: Scan mode configuration

3.5 Constraints

3.5.1 Sun optimized

There are two cases in Sun-optimizing: minimum and maximum angular separation from the Sun vector. The instrument that should be closest to the Sun is the solar array,

meanwhile, most of the other instruments and the radiators should stay the furthest from the Sun to prevent being harmed by the Solar flux.

Let β the angle between $\vec{pl}_1$ and $\vec{pl}_2$. Since these two payloads are fixed in the satellite, β is unchanged and can be calculated as:

$$\beta = \cos^{-1}(\vec{pl}_1 \cdot \vec{pl}_2)$$

Let q_1 the align quaternion of $\vec{pl}_1$ to $\vec{los}_1$. After aligning the first primary payload to the first LoS, $\vec{los}_1$ is identical to the $\vec{pl}_1$, like Figure 14 - left. The $\vec{pl}_2$ now has the value $\vec{pl}_2'$ due to this alignment, with

$$\vec{pl}_2' = q \otimes \vec{pl}_2 \otimes q^*$$

and $\vec{pl}_2'$ is now expressed in ICRF.

The Sun vector extracted from section 3.4.3 will be the $\vec{los}_2$.

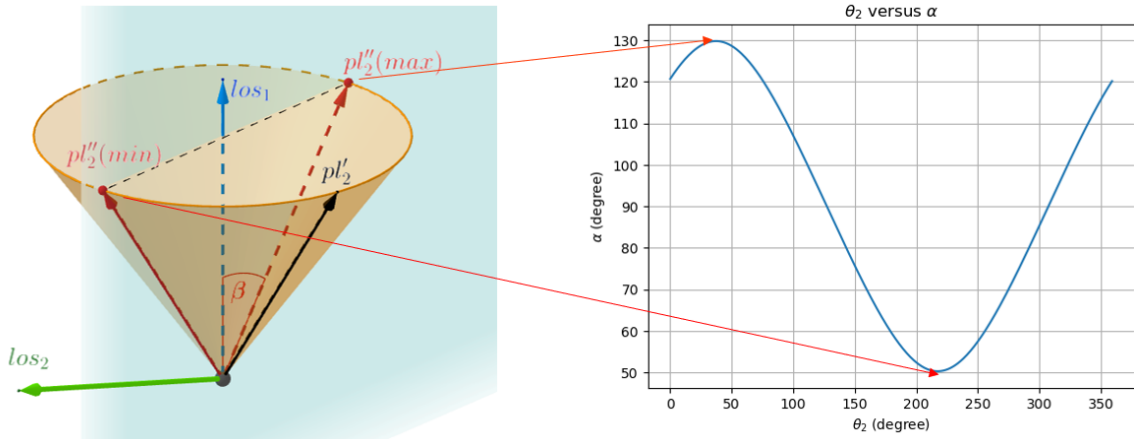


Figure 14: Rotation of $\vec{pl}_2$ about $\vec{pl}_1$

Let $\vec{pl}_2''$ the final position of the secondary payload, and α the angle between the $\vec{pl}_2''$ to the $\vec{los}_2$, thus $\alpha = \cos^{-1}(\vec{pl}_2'' \cdot \vec{los}_2)$. This α angle is the angle that requires optimization. The $\vec{pl}_2''$ always lies on a right circular cone, which takes the $\vec{pl}_1$ as the height, and β is the half-angle of the cone. A graph representing the relation between θ versus α is plotted in Figure 14 - right, with $\theta = [0; 2\pi]$ the position of the $\vec{pl}_2'$ on the base of that cone, and $\theta = 0$ is when $\vec{pl}_2'' \equiv \vec{pl}_2'$.

It can be observed that the graph in Figure 14-right displays a sinusoidal pattern, with one maximum and one minimum value of α , corresponding to two positions of $\vec{pl}_2''$, and

these two vectors lie on the same plane with $\overrightarrow{los_1}$ and $\overrightarrow{los_2}$. This graph does not have the maximum at 180 degrees and minimum at 0 degrees, due to the angle constraint between the two payloads. Since these optima occurs when the $\overrightarrow{pl_2''}$ is coplanar with $\overrightarrow{los_1}$ and $\overrightarrow{los_2}$, the position of the $\overrightarrow{pl_2''}$ can be found on this plane.

Let's define the vector $\vec{b}$:

$$\vec{b} = (\overrightarrow{los_1} \wedge \overrightarrow{los_2}) \wedge \overrightarrow{los_1}$$

Vector $\vec{b}$ then be normalized and will be perpendicular to $\overrightarrow{los_1}$. And hence, $\overrightarrow{pl_2''}$ can be expressed using $\overrightarrow{los_1}$ and $\vec{b}$ as follow:

$$\overrightarrow{pl_2''(max)} = \cos(\beta) \overrightarrow{los_1} - \sin(\beta) \vec{b}$$

$$\overrightarrow{pl_2''(min)} = \cos(\beta) \overrightarrow{los_1} + \sin(\beta) \vec{b}$$

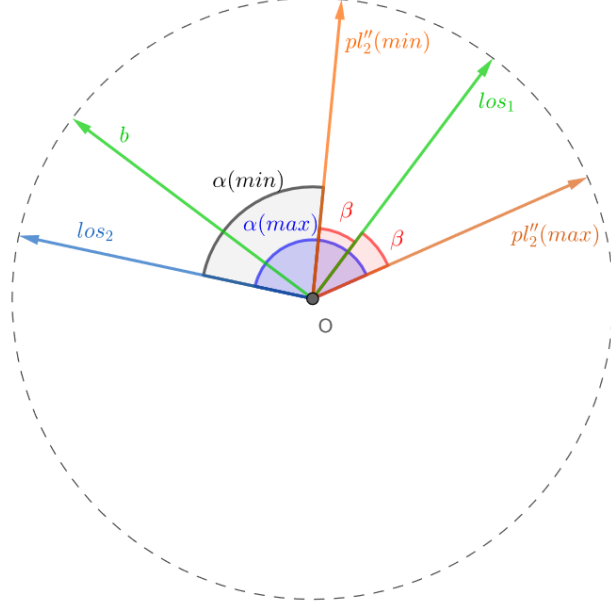


Figure 15: Optima position of $\overrightarrow{pl_2''}$

After finding the desired coordinate of the secondary payload, a quaternion can be created to transform the $\overrightarrow{pl_2'}$ to the desired $\overrightarrow{pl_2''}$, by using the equations in section 3.1.2.

3.5.2 Rotation with period

At time $t_0 = 0$, the attitude of the satellite is q_0 with respect to the ICRF. The user wants it to be q_0 again after the minimum time of P . This P is the period of the rotation of the secondary payload around the primary payload. Let's consider the time t after t_0 . Assume that at t_0 , coordinate of the secondary payload is $\overrightarrow{pl_2'}$, and at time t_1 , coordinate of the secondary payload is $\overrightarrow{pl_2''}$. $\vec{v}'$ and $\vec{v}''$ are consecutively the projection of those two vectors onto the base of the cone in Figure 14 - left. The $\overrightarrow{pl_1'}$ is pointing out of the page.

The angle separation of $\vec{v}'$ and $\vec{v}''$ is θ , and:

$$\theta = \text{mod}\left(\frac{t - t_0}{P}, 2\pi\right)$$

After finding this angle of rotation θ , a quaternion can be constructed to present this

rotation, using Equation 2.

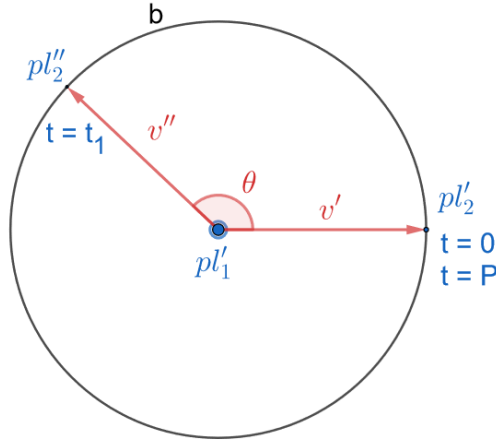


Figure 16: Rotation with fixed period of $\vec{pl'_2}$ about $\vec{pl'_1}$

3.6 Integration of quaternion

Let q_1 the align quaternion of $\vec{pl'_1}$ to $\vec{los_1}$, and q_2 the align quaternion of the $\vec{pl'_2}$ and the desired vector $\vec{los_2}$. The final quaternion that perform two consecutively the rotation by q_1 and then q_2 is:

$$q = q_2 \otimes q_1 \quad (8)$$

4 Results

In this section, the inputs and outputs file will be described, to demonstrate more clearly the workflow in section 2. A case study will also be conducted to present the ability of DOCKS' Quaternions.

4.1 Inputs

Figure 17 shows an example of a YAML file as the input to DOCKS' Quaternions. The trajectory file, the payload vectors, the modes and constraints, and the output directory are stated in the YAML file. The payload vectors are not necessarily normalized, because the module will automatically perform the normalization of vectors. All the modes and constraints have been described in the previous section of this report. The user can choose one mode for the primary payload and one constraint for the secondary payload.

The request parameters for the modes and constraints are optional because some kinds of LoS do not require any parameters (e.g. nadir pointing, Sun-optimized). The time format in the YAML file is ISO-8601, which displays the time format “YYYY-MM-DDTHH:MM:SS”. The DOCKS’s Quaternions will inherit the time system from the trajectory file, as depicted in section 3.4, and the time input in the YAML file should also follow the same time system to avoid any timing issues. The recommended time scale is Barycentric Dynamical Time (TDB) since the Propagator’s trajectory output uses TDB as the time scale. More information about the time scale can be found in Appendix 7.3.

```
# =====Trajectory file
traj_file: 'inputs/input_trajectory.txt'
#=====Pointing strategies
strategies:
  stg0:
    epoch: '2011-01-01T23:30:00'
    end:
    payload_1_vector : [1,0,1]
    payload_2_vector : [0,-1,0]
    payload_1_mode: 'nadir'
    mode_1_request:
    payload_2_mode: 'rotation'
    mode_2_request: 5087
  stg1:
    epoch: '2011-01-02T23:30:00'
    end: '2011-01-03T00:30:00'
    payload_1_vector : [0,0,1]
    payload_2_vector : [0,0.5,0.866]
    payload_1_mode: 'scan'
    mode_1_request: [20,0]
    payload_2_mode: 'sun_optimized_closest'
    mode_2_request:

#===== outputs directory
File_name: 'output_AEM.txt'
Output_directory: '../Downloads/VTS/Data/DOCKS'
```

Figure 17: Example of input YAML file

payload_1_mode	mode_1_request	Units	Example
nadir	None	None	None
inertial	[RA, Dec]	[degree, degree]	[27, 15]
limb_track	[κ , altitude]	[degree, kilometer]	[30, 50]
target_pointing	[Latitude, Longitude, altitude]	[degree, degree, kilometer]	[48.2, 2.23, 0.165]
sun	None	None	None
to_body	Solar System Body	None	'VENUS'
scan	[γ , κ]	[degree, degree]	[20, 90]
user_define_vector	None	None	[0.5, 0, 0.866]

Table 5: Available modes and corresponding parameters for primary payload

payload_2_mode	mode_2_request	Units	Example
sun_optimized_furthest	None	None	None
sun_optimized_closest	None	None	None
rotation	Period	seconds	5807

Table 6: Available constraints and corresponding parameters for secondary payload

4.2 Outputs

```

CIC_AEM_VERSION = 1.0
CREATION_DATE = 2023-08-10 17:34:27.359169
ORIGINATOR = CENSUS / LabEx ESEP - Paris Observatory - PSL University Paris

META_START

OBJECT_NAME = DOCKS_Sat
OBJECT_ID = CORPS
REF_FRAME_A = ICRF
REF_FRAME_B = SC_BODY_1
ATTITUDE_DIR = A2B
TIME_SYSTEM = TDB
ATTITUDE_TYPE = QUATERNION
QUATERNION_TYPE = FIRST

META_STOP

59215 0.00000 0.6421100015 0.0000000000 0.0000000000 -0.7666125136
59215 10.00000 0.6421112310 0.0000000000 0.0000000000 -0.7666114838
59215 20.00000 0.6421124583 0.0000000000 0.0000000000 -0.7666104557
59215 30.00000 0.6421136835 0.0000000000 0.0000000000 -0.7666094295
59215 40.00000 0.6421149066 0.0000000000 0.0000000000 -0.7666084051
59215 50.00000 0.6421161275 0.0000000000 0.0000000000 -0.7666073824
59215 60.00000 0.6421173463 0.0000000000 0.0000000000 -0.7666063615
59215 70.00000 0.6421185630 0.0000000000 0.0000000000 -0.7666053424
59215 80.00000 0.6421197775 0.0000000000 0.0000000000 -0.7666043251

```

Figure 18: Example of AEM output file

Figure 18 is the output text file that contains the calculated quaternions due to the request from the inputs. The header contains the information about the quaternion file: The

type of attitude contained in this file follows scalar-first quaternion, and they present the quaternion transformation from ICRF to the spacecraft’s body. The time system is Barycentric Dynamical Time (TDB), the same as the trajectory file in section 3.4. The first two columns of the data are identical to that of the trajectory file. The four remaining columns are the values of four parameters of the quaternion, subsequently q_0 , q_1 , q_2 , and q_3 .

4.3 Case study with Venus Express

Venus Express (VEX) [25] is chosen to be a study case, thanks to the numerous payloads and pointing strategies that this mission provides [25, 26]. The payloads and the corresponding boresights with respect to the spacecraft frame can be found in Table 7. Each payload has its own frame (see Appendix 7.2), which may or may not aligned with the satellite’s frame, and it is noteworthy that the payload vectors are just the boresight of the instrument’s frame and are usually aligned to the +Z axis of the instrument frame. The information about the boresight vector is not sufficient for the whole frame representation of the instrument (i.e. the boresight and the Field of View). The information about the payload and instrument’s frames can be found in the frame kernels (fk)[21, 26], extracted from NAIF’s SPICE.

Instrument	Name	Payload vectors (w.r.t s/c’s frame)
ASPERA	Analysers of Space Plasma and Energetic Atoms	[0, -1, 0]
MAG	Venus Express Magnetometer	[0, 0, 1]
PFS	Planetary Fourier Spectrometer	[0, 0, 1]
SPICAV/SOIR	Ultraviolet and Infrared Atmospheric Spectrometer	[0, 0, 1]
VIRTIS	Ultraviolet/visible/near-infrared Mapping Spectrometer	[0, 0, 1]
VMC	Venus Monitoring Camera	[0, 0, 1]
HGA1	High Gain Antenna 1	[0.996, 0, 0.0877]
HGA2	High Gain Antenna 2	[-0.996, 0, 0.0877]

Table 7: VEX’s payloads

The term “phase” in this report’s context refers to an interval of time where specific modes of the primary and secondary payload are requested to be performed. During the operation of the satellite, each phase of the mission requires specific operations of payloads.

No.	Phase	Instruments	$\vec{pl}_1$	$\vec{pl}_1$ mode and requests (if any)	$\vec{pl}_2$	$\vec{pl}_2$ mode and requests (if any)
1	Nadir	VMC	[0, 0, 1]	'nadir'	[0,-1,0]	'sun_optimized_furthest'
2	Limb track	VIRTIS	[0,0,1]	'limb_track', [20, 0]	[0,-1,0]	'sun_optimized_furthest'
3	Star Occultation	SPICAV	[0,0,1]	'inertial_pointing' [201.3, - 11.15]	[0, -1, 0]	'sun_optimized_furthest'
4	Earth Pointing	HGA2	[-0.996, 0, 0.088]	'to_body', 'EARTH'	[0,0,1]	'sun_optimized_furthest'

No	Epoch time	End time
1	'2011-01-01T23:30:00.000'	'2011-01-02T00:30:00.000'
2	'2011-01-02T23:30:00.000'	'2011-01-03T00:30:00.000'
3	'2011-01-04T23:30:00.000'	'2011-01-05T00:30:00.000'
4	'2011-01-06T05:00:00.000'	'2011-01-06T06:00:00.000'

Table 8: Examples of pointing strategies for VEX

Since the SPICAV instrument takes the hot UV stars to be the referenced stars, in the Star Occultation observation phase, the Spica star is chosen, with the Right Ascension of 201.3° and Declination of -11.15°. The secondary payload vector is the boresight of ASPERA and the radiators for phases numbered 1,2,3; and in phase 4, it is the boresight of most of the instruments. The secondary payloads are set to be as far as possible from the Sun to protect the instruments from the high radiation. In limb tracking mode, the altitude from the surface is 0, i.e. the satellite points to the horizon; meanwhile the angle κ is chosen arbitrarily as 20 degrees. In Earth pointing mode, at the time of simulation (January 2011), Venus was close to Earth (the Venus - Sun - Earth angle is smaller than 90 degrees), as shown in Figure 19, hence the second High Gain Antenna HGA2 will be used for transmission to Earth [25].

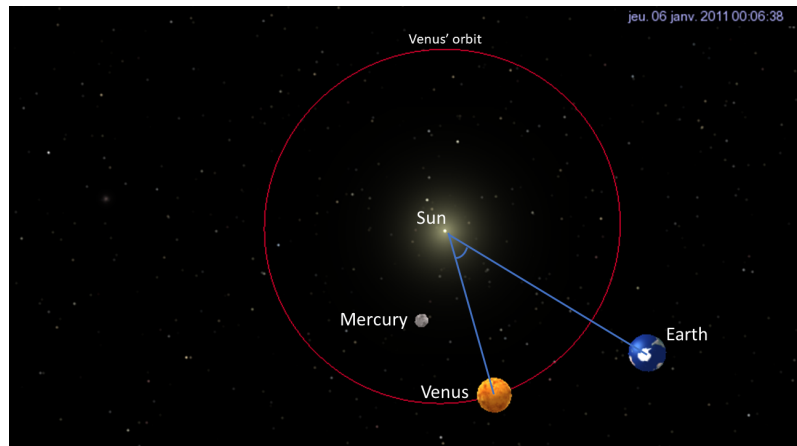


Figure 19: Venus - Sun - Earth angle at the beginning of January 2011


```

#===== Trajectory file
traj_file: 'inputs/output_traj_vex.txt'

#=====Pointing strategies
strategies:
  stg1:
    epoch: '2011-01-01T23:30:00'
    end: '2011-01-02T00:30:00'
    payload_1_vector : [0,0,1]
    payload_2_vector : [0,-1,0]
    payload_1_mode: 'nadir'
    mode_1_request:
    payload_2_mode: 'sun_optimized_furthest'
    mode_2_request:
  stg2:
    epoch: '2011-01-02T23:30:00'
    end: '2011-01-03T00:30:00'
    payload_1_vector : [0,0,1]
    payload_2_vector : [0,-1,0]
    payload_1_mode: 'limb_track'
    mode_1_request: [20,0]
    payload_2_mode: 'sun_optimized_furthest'
    mode_2_request:
  stg3:
    epoch: '2011-01-04T23:30:00'
    end: '2011-01-05T00:30:00'
    payload_1_vector : [0,0,1]
    payload_2_vector : [0,-1,0]
    payload_1_mode: 'inertial_pointing'
    mode_1_request: [201.3, -11.15]
    payload_2_mode: 'sun_optimized_furthest'
    mode_2_request:
  stg4:
    epoch: '2011-01-06T05:00:00'
    end: '2011-01-06T06:00:00'
    payload_1_vector : [-0.996,0,0.088]
    payload_2_vector : [0,0,1]
    payload_1_mode: 'to_body'
    mode_1_request: 'EARTH'
    payload_2_mode: 'sun_optimized_furthest'
    mode_2_request:

#===== outputs directory
File_name: 'output_AEM.txt'
Output_directory: '/../VTS/Data/VEX'

```

Figure 20: Input YAML file for VEX study case

Thanks to DOCK's Propagator, a trajectory ephemeris of the VEX mission can be created. In the operation lifetime of VEX, VEX orbited Venus in a highly elliptical orbit (eccentricity of 0.839) with altitude at periapsis of 250 km, and inclination of 90 degrees [25]. Epoch time of simulation is chosen at noon of January 1st, 2011 (UTC), at the periapsis, since this was the time that VEX was in operation. Except for the Earth Pointing Phase, the epoch times and end times are chosen at 23h30 and 00h30 of the next day because these are the intervals that VEX was close to Venus, thus they are good for atmospheric observation of Venus. On the other hand, the Earth Pointing Phase does not require close distance to Venus' surface, hence this phase can be performed from 5 a.m. to 9 p.m. on the same day. In this simulation, the interval for Earth Pointing mode is chosen to be one hour, from 5 a.m. to 6 a.m. of January 6th 2011.

With the input files as in Figure 20, DOCKS' Quaternions calculates the attitude and generates an AEM text file. This text can be read by VTS. The result is shown in Figure 21, 22, 23 and 24. The scale of objects in these figures is exaggerated so that the pointing of the payloads can be noticed. The payload vectors are expressed by elliptical cones with a half-angle around the two directions is 0.5 degrees. The boresight of VMC, VIRTIS, and SPICAV, with the value [0,0,1], is indicated by the red cone and can be seen aligning with the +Z vector in the satellite frames. The vector [0,-1,0] is represented by the purple cone, and the HGA2's boresight is indicated by the cyan cone and only appears in the Earth pointing mode. The yellow vector is the Sun direction vector and

the purple vector is the body vector, pointing to the center of the body. VTS also has a plot of the AEM file (the lower plot in the figures), which plots the four parameters of the quaternion in the file with respect to time. Four components of the quaternion q_0 , q_1 , q_2 , q_3 are respectively colored blue, red, green, yellow.

4.3.1 Phase 1: Observation, Nadir Pointing

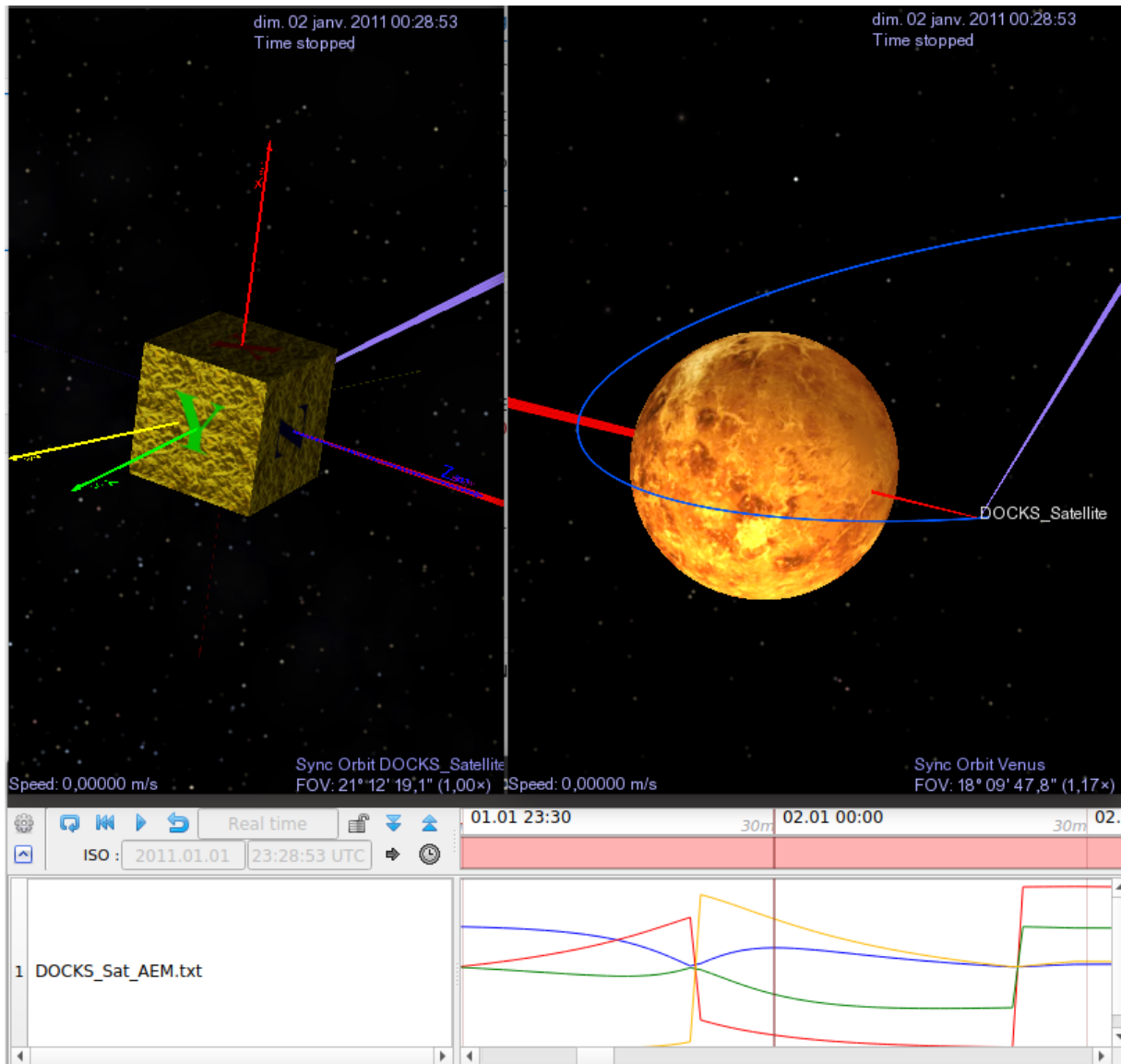


Figure 21: Observation mode, Nadir Pointing

The VMC's boresight is aligned with the body vector, hence it is pointed in the nadir direction. The -Y panel is far from the Sun. It is observed in the graph that there is a change of sign at 2011-01-01T23:57:00, that is because of the convention that q_0 is kept non-negative. This change of sign does not mean a sudden change of attitude, due

to the characteristics of quaternion: $q = -q$. Since the satellite in this time interval is close to Venus, the nadir vector changes rapidly with time, which leads to rapid change of quaternion.

4.3.2 Phase 2: Observation, Limb tracking

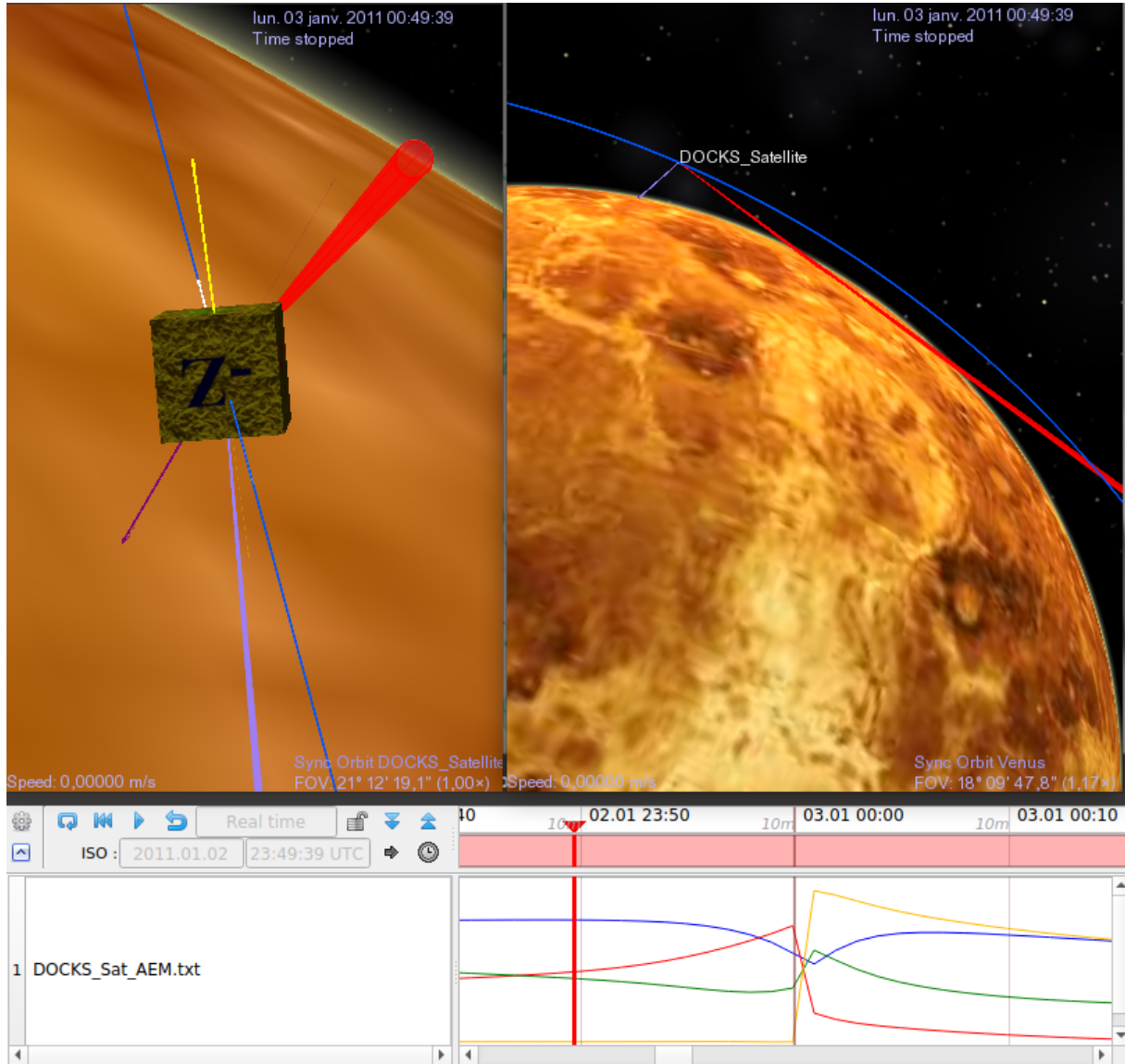


Figure 22: Observation mode, Limb tracking

The VIRTIS' boresight tracks the limb of Venus at an altitude of 0km and makes with the velocity vector (the white vector tangent to the orbit path) an angle of 20 degrees. The -Y panel is far from the Sun direction. Like the nadir phase, a robust change at 2011-01-03T00:00:00 can be noticed, due to the Sun-optimized algorithm. This change of angular separation between the Sun vector and the secondary payload vector can be

seen in Figure 28, nonetheless, the tracking along the limb is smooth and continuous. At about 2011-01-03T00:01:00, there is a change of sign to keep q_0 positive, as explained in the nadir phase. The quaternion also changes rapidly with respect to time, due to the fast change of the nadir vector (presented in Figure 25 and 26).

4.3.3 Phase 3: Observation, Star Occultation

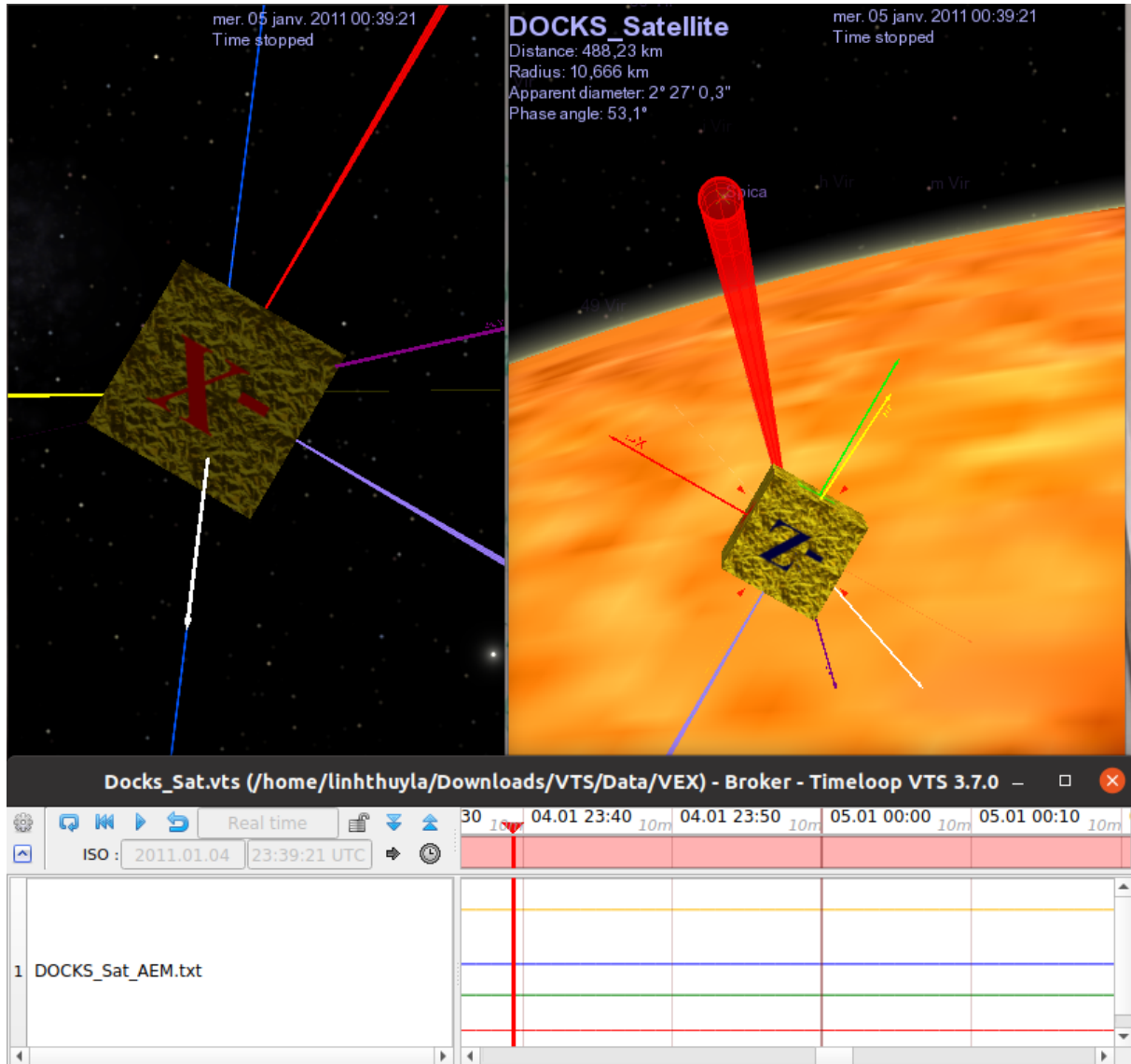


Figure 23: Observation mode, Star Occultation

In the inertial pointing mode, the SPICAV's boresight looks at the Spica star. At this moment, Spica is not yet occulted by Venus, but the LoS from the satellite to the star was going through the atmosphere of Venus. Unlike the two previous phases, it is observed that the attitude profile does not change very much. The reason for this phenomenon is

the pointing vector (inertial pointing) and the Sun vector change insignificantly in such a small interval (one hour) (presented in Figure 25 and 26).

4.3.4 Phase 4: Transmission, Earth Pointing

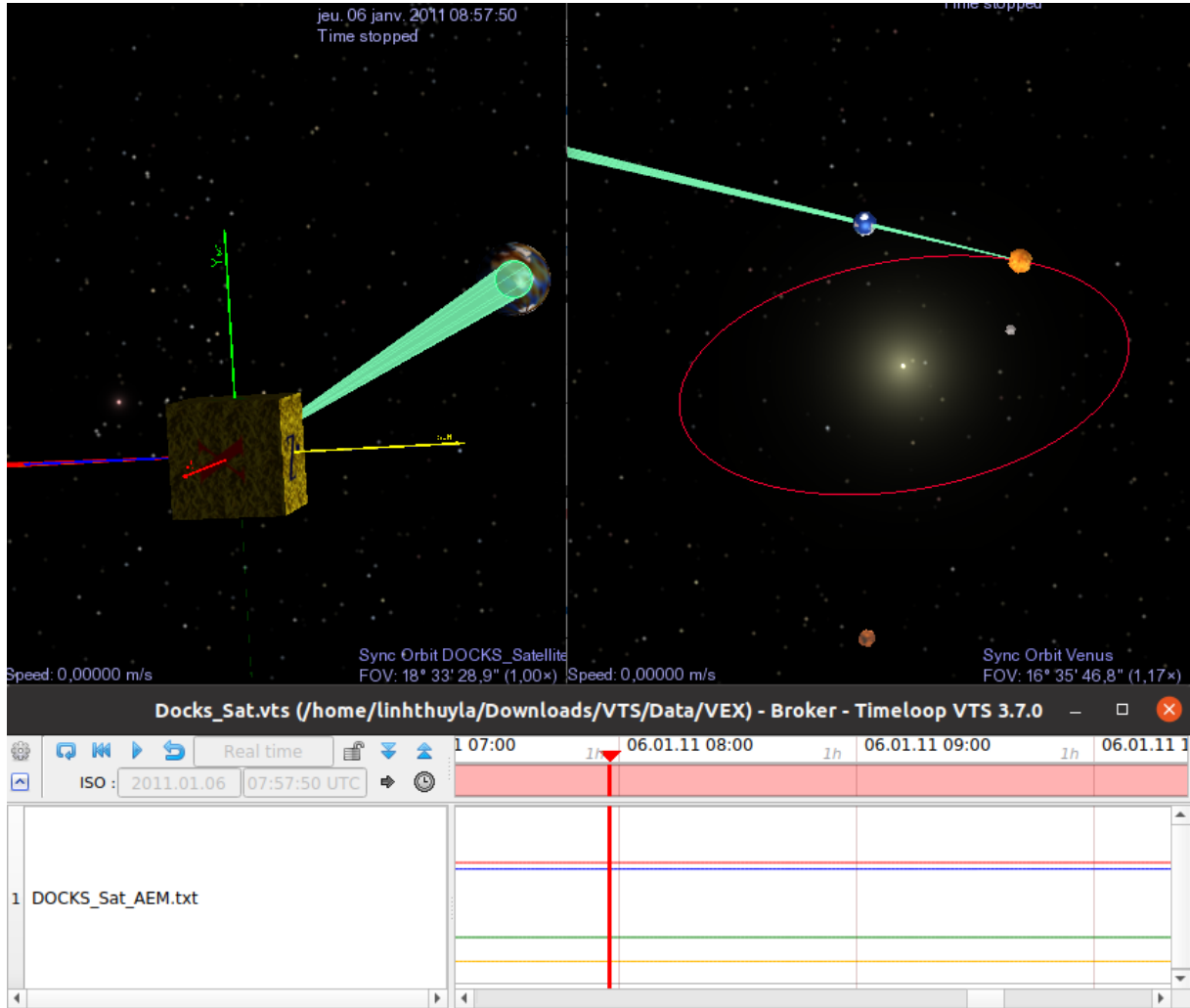


Figure 24: Earth Pointing mode

Similar to the inertial pointing phase, the pointing vector (Venus-Earth vector) and the Sun vector change insignificantly in such a small interval (one hour), hence the attitude profile does not change much with time (as presented in Figure 25 and 26).

4.3.5 Investigation on the case study

From the algorithms in section 3.4, the Line of Sight profile for each phase is generated and represented in Figure 25. In the nadir (phase 1) and limb tracking phase (phase 2), the primary payload vector $\vec{pl}_1$ changes rapidly because the satellite's position to the

body changes rapidly within that interval, and the formula to build the LoS in these two cases involves the nadir vector. In the inertial pointing phase (phase 3), $\vec{pl}_1$ remains unchanged because the target star is fixed in ICRF. Meanwhile, in the Earth pointing mode (phase 4), $\vec{pl}_1$ does change with time since Venus and Earth orbit the Sun and the satellite orbits Venus, however, the interval in the plot is short (one hour), hence the change seems insignificant (as presented in Figure 25 and 26).

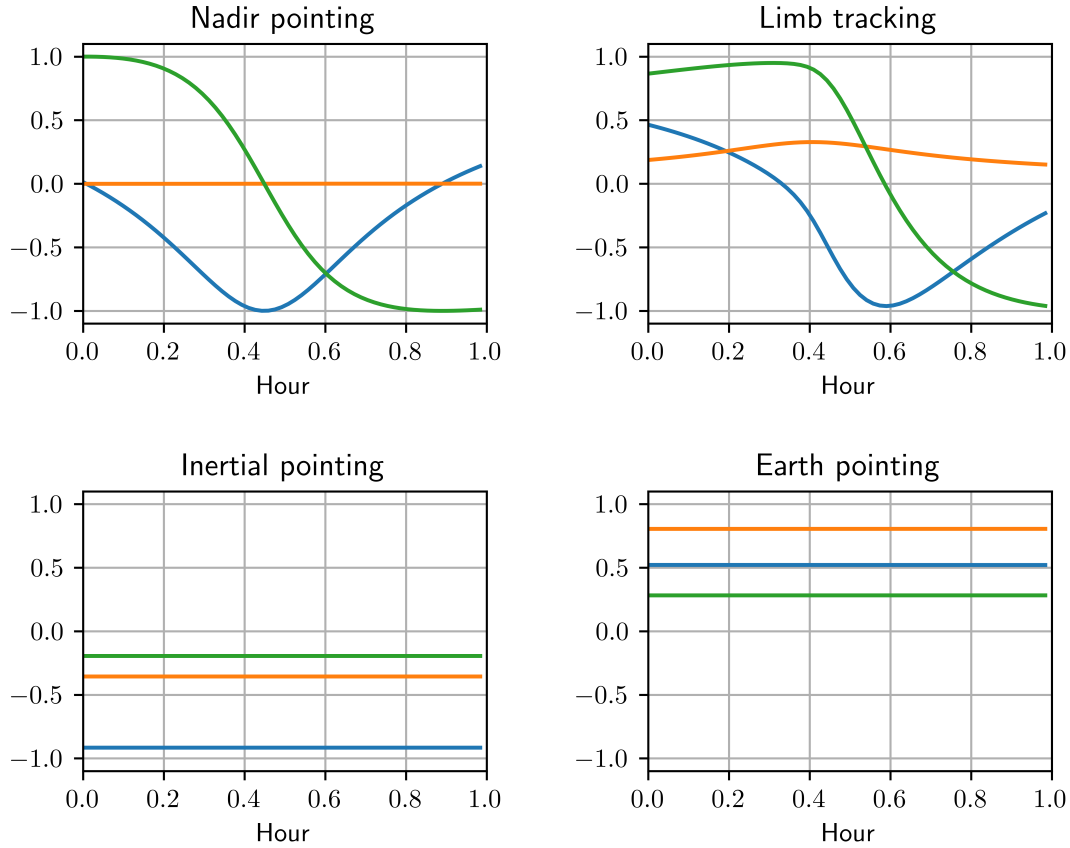


Figure 25: Primary LoS in ICRF (x value: blue, y value : orange, z value: green)

The secondary LoS in all four cases is the Sun vector since the set constraint is Sun-optimized. The change of the Sun vector in one hour might not be noticeable, but a gradual change from the upper left subfigure to the lower right subfigure can be detected in Figure 26, because the gap between these two phases is 5 days (from January 1st to January 6th), corresponding to a change of 8 degrees in the orbit of Venus around the Sun (a year of Venus takes only about 225 Earth days).

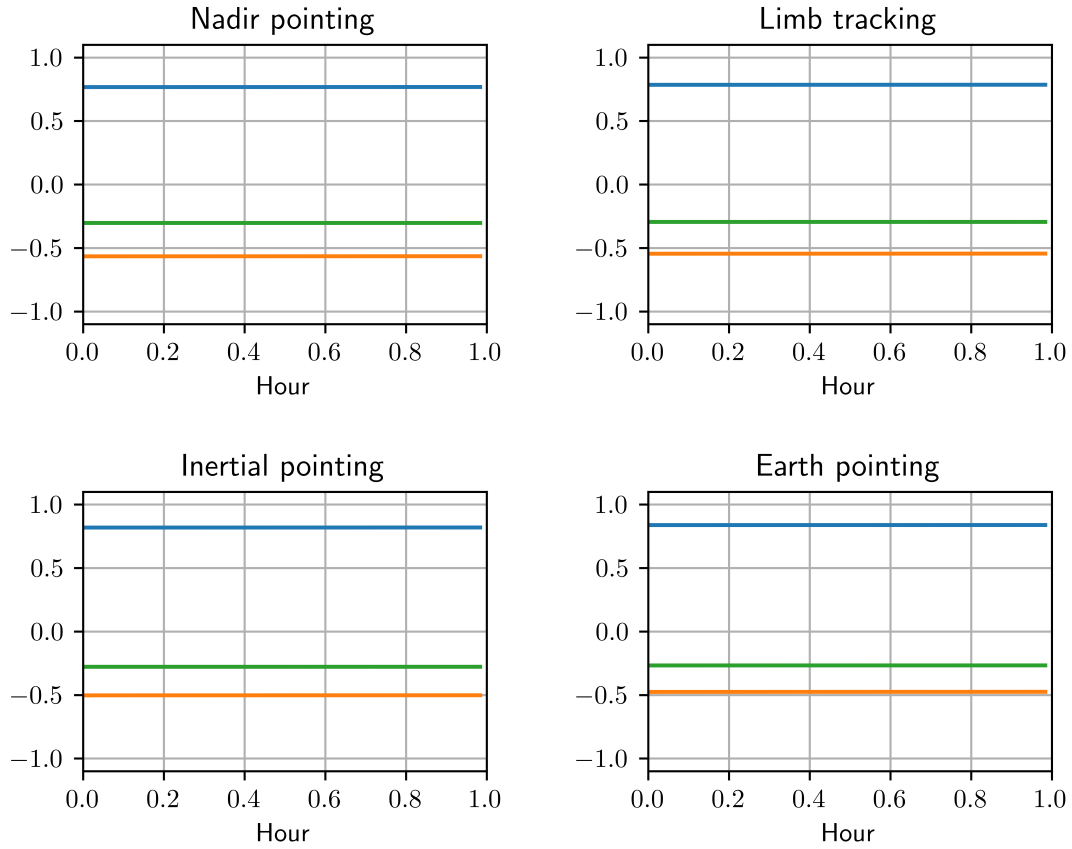


Figure 26: Sun vector in ICRF (x value: blue, y value : orange, z value: green)

To verify the accuracy of the pointing and to check whether it meets the user's request, the angles between the payload vectors and the Lines of Sights are investigated.

After rotation by the quaternion q generated by DOCKS' Quaternions, the pointing of primary payload vector in ICRF $\overrightarrow{pl_{1,icrf}}$ can be obtained by:

$$\overrightarrow{pl_{1,icrf}} = q \otimes \overrightarrow{pl_1} \otimes q^*$$

Since the primary LoS $\overrightarrow{los_1}$ and $\overrightarrow{pl_{1,icrf}}$ are both presented in the ICRF, the angle of these two vectors can be calculated by :

$$\varphi = \arccos(\overrightarrow{pl_{1,icrf}} \cdot \overrightarrow{los_1})$$

Figure 27 shows value of φ versus interval of simulation (one hour). It is marked that the $\overrightarrow{pl_{1,icrf}}$ and $\overrightarrow{los_1}$ are very well aligned in all cases, with the angle φ is 0.0 or very close to 0. Hence, the primary mission is achieved.

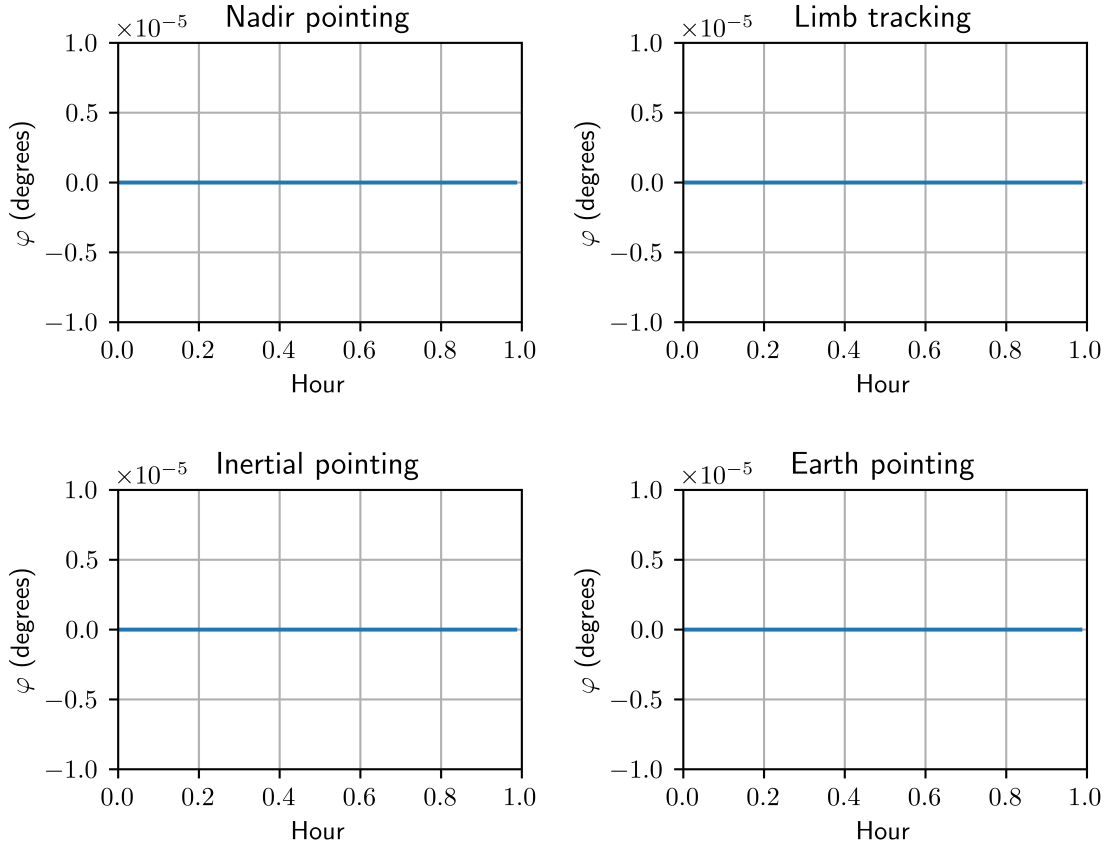


Figure 27: Angle between primary payload vector and primary LoS

Similar to the primary payload, the angle between the secondary payload and secondary LoS can be calculated as:

$$\overrightarrow{pl_{2,icrf}} = q \otimes \overrightarrow{pl_2} \otimes q^*$$

$$\theta = \arccos(\overrightarrow{pl_{2,icrf}} \cdot \overrightarrow{los_2})$$

The secondary LoS in the example is the Sun vector because the modes of the secondary payload are set to be Sun-optimized. In all phases, the angular separation between $\overrightarrow{pl_{2,icrf}}$ and $\overrightarrow{pl_2}$ (denoted as θ) never falls below 90 degrees, which means $\overrightarrow{pl_2}$ is well-kept to be far from the Sun and the sunlight won't affect the field of view of the instrument having this boresight.

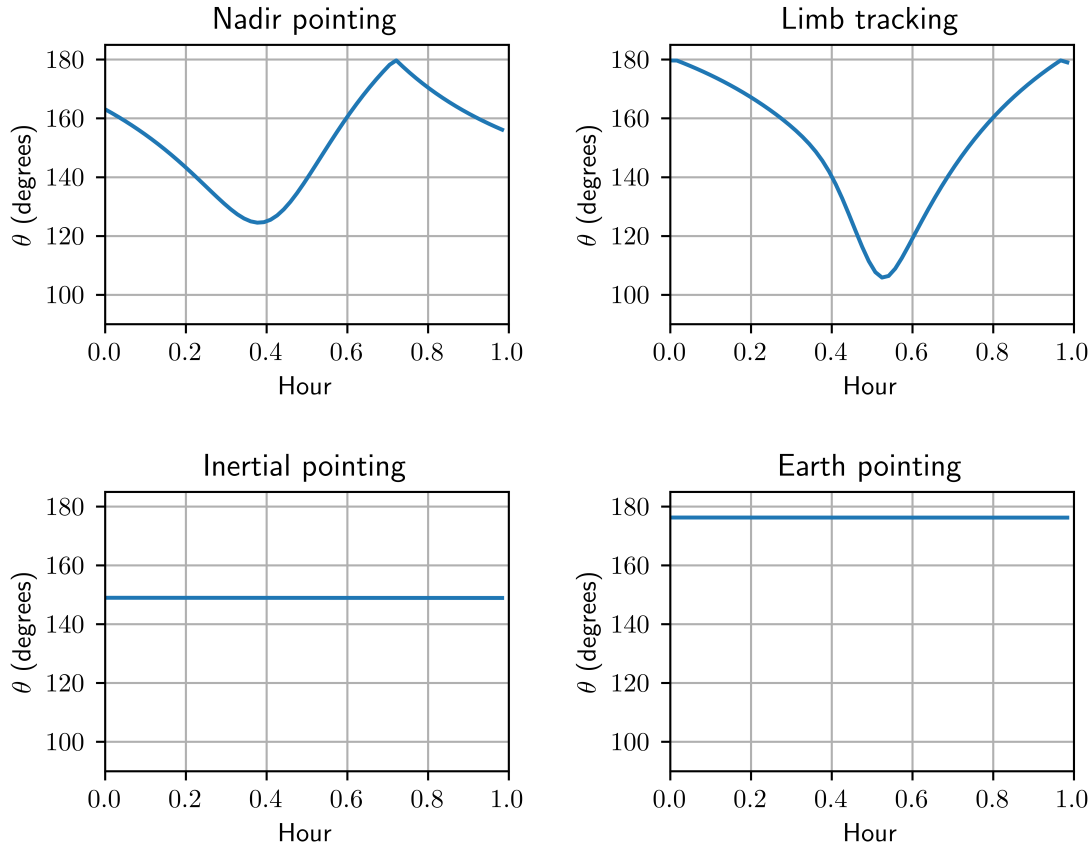


Figure 28: Angle between secondary payload vector and secondary LoS (Sun vector)

A big different pattern of θ can be noticed among the four cases. θ in phase 3 and phase 4 remain almost unchanged during the simulation time, respectively at 148 degrees and 176 degrees. Meanwhile, the graph of θ in phase 1 and phase 2 is more complicated: the largest achievable value of θ is 180 degrees, which means opposite to the Sun direction; and the smallest maximum separation angle is 125 degrees and 110 degrees, respectively. The divergence of trend between the phases can be explained by the trend of primary LoS. In phases 1 and 2, the LoS relates closely to the nadir vector, and the rapid change of position of the satellite in the orbit leads to the rapid change of the primary vector in ICRF. Since the $\overrightarrow{los_1}$ is taken to be the rotational axis for the rotation of the secondary payload, leads to the rapid change of the second quaternion, as described in section 3.5.1. Hence, although the corresponding LoS (Sun vector) barely changes, the pointing of the secondary payloads and its angular separation with the Sun vector still evolves with time and the $\overrightarrow{los_1}$. It is noteworthy that the angle θ is not periodic and depends strongly on the primary target, the position, and the angle between the two

payload vectors.

5 Discussions

After a few months, a prototype of DOCKS' Quaternions/ Pointing Strategies has been established with certain features depicted throughout the report. DOCKS' Quaternions so far has succeeded in dealing with only two payload vectors, inferred in 3.3, which can be a common case for nanosatellite missions; yet a more complex mission can have more than two payloads with additional pointing strategies and constraints. The user, under these circumstances, must prioritize the payloads manually to make use of DOCKS's Quaternion.

Since this module is still under construction, many of the module's features can be added or improved in the future. All of these modifications will be made to benefit the user and mission planner's experience.

- more modes and constraints are considered to be included (e.g. constraint: keep two vectors in the specific cones);
- Field of View of the payload can be taken into account to choose the right attitude;
- a more effective configuration for the input file is also under investigation (e.g. shorten the format, auto-generated YAML file, etc.)
- produces an additional ephemeris file to help the user choose the time slot (e.g. file contains the angle separation of Sun vector with a payload's boresight)

6 Conclusions

Up to now, the DOCKS' Quaternions succeeded in generating attitude ephemeris for compact yet pragmatic pointing strategies of satellites, especially for nanosatellites. The defined built-in pointing strategies in the module are meticulously selected from the investigation of plenty of nanosatellite missions, thus the module can satisfy several cases. The design of specific pointing strategies within the module is carried out with the implementation of geometry, quaternion algebra, and optimization algorithms. The module's user interface has been designed to provide users with access to specifying their desired pointing parameters: setting the time interval, configuring their satellite's

payloads, selecting the pointing strategies, and defining the corresponding parameters. Once obtaining attitude and orbit ephemeris, users can use VTS to gain insights into the satellite's profile at any point in the mission's lifetime. This will aid mission planners in making decisions about when to schedule tasks and which pointing strategies to employ, maximizing the satellite's ability throughout the mission. The development of DOCKS' Quaternions module is still ongoing, with new pointing strategies and features planned.

References

- [1] Consultative Committee for Space Data Systems (CCSDS). *MISSION PLANNING AND SCHEDULING*. URL: <https://public.ccsds.org/Pubs/529x0g1.pdf>. (accessed: 20.08.2023).
- [2] James Richard Wertz et al. *Space mission analysis and design*. Springer, 1999.
- [3] Consultative Committee for Space Data Systems (CCSDS). *POINTING REQUEST MESSAGE*. URL: <https://public.ccsds.org/Pubs/509x0b1c1.pdf>. (accessed: 20.08.2023).
- [4] Consultative Committee for Space Data Systems (CCSDS). *ORBIT DATA MESSAGES*. URL: <https://public.ccsds.org/Pubs/502x0b3e1.pdf>. (accessed: 20.08.2023).
- [5] Consultative Committee for Space Data Systems (CCSDS). *ATTITUDE DATA MESSAGES*. URL: <https://public.ccsds.org/Pubs/504x0b1c1.pdf>. (accessed: 20.08.2023).
- [6] CENSUS. *CENSUS*. URL: <https://census.psl.eu/>. (accessed: 20.08.2023).
- [7] CENSUS. *DOCKS*. URL: <https://census.psl.eu/spip.php?rubrique37#archiveprototypagesavecdocks>. (accessed: 20.08.2023).
- [8] Rashika Jain and Boris Segret. *DOCKS, an open-source software suite for space mission profiles*. URL: https://events.libre.space/event/5/contributions/162/attachments/103/130/session%201-4%20JAIN%20DOCKS%40CENSUS%20COSCOW_2021.pdf. (accessed: 20.08.2023).
- [9] Erik Kulu. *Nanosats Database*. URL: <https://www.nanosats.eu/>. (accessed: 27.08.2023).
- [10] Erik Kulu. "Nanosatellite Launch Forecasts-Track Record and Latest Prediction". In: (2022).
- [11] Rashika Jain, Harshul Sharma, and Boris Segret. "DOCKS Propagator: An Open-source Adaptive Time-step Trajectory Propagator for CubeSat Missions". In: *2022 IEEE Aerospace Conference (AERO)*. IEEE. 2022, pp. 1–12.
- [12] James R Wertz. *Spacecraft attitude determination and control*. Vol. 73. Springer Science & Business Media, 2012.
- [13] Hannes Sommer et al. "Why and how to avoid the flipped quaternion multiplication". In: *Aerospace 5.3* (2018), p. 72.
- [14] NAIF. *Quaternion White Paper*. URL: https://naif.jpl.nasa.gov/pub/naif/misc/Quaternion_White_Paper/Quaternions_White_Paper.pdf. (accessed: 20.08.2023).

- [15] VTS. *Visualization Tool for Space Data*. URL: <https://timeloop.fr/uts/>. (accessed: 31.08.2023).
- [16] Hanspeter Schaub. *Kinematics: Describing the Motions of Spacecraft*. URL: <https://www.coursera.org/specializations/spacecraft-dynamics-control>. (accessed: 03.09.2023).
- [17] Kieran Wynn. *pyquaternion*. URL: <http://kieranwynn.github.io/pyquaternion/>. (accessed: 22.05.2023).
- [18] Numpy. *Quaternions in numpy*. URL: <http://kieranwynn.github.io/pyquaternion/>. (accessed: 22.05.2023).
- [19] SciPy. *scipy.spatial.transform.Rotation.as_matrix*. URL: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.html>. (accessed: 22.05.2023).
- [20] NAIF. *Orientation or Attitude Kernel*. URL: https://naif.jpl.nasa.gov/pub/naif/toolkit_docs/Tutorials/pdf/individual_docs/20_ck.pdf. (accessed: 30.08.2023).
- [21] NAIF. *Frames Kernel*. URL: https://naif.jpl.nasa.gov/pub/naif/toolkit_docs/Tutorials/pdf/individual_docs/21_fk.pdf. (accessed: 30.08.2023).
- [22] NAIF. *Ephemeris Subsystem*. URL: https://naif.jpl.nasa.gov/pub/naif/toolkit_docs/Tutorials/pdf/individual_docs/18_spk.pdf. (accessed: 30.08.2023).
- [23] IMCCE - Observatoire de Paris. *Institut de mécanique céleste et de calcul des éphémérides*. URL: <https://www.imcce.fr/>. (accessed: 30.08.2023).
- [24] NAIF. *Planetary Constants Kernel*. URL: https://naif.jpl.nasa.gov/pub/naif/toolkit_docs/Tutorials/pdf/individual_docs/19_pck.pdf. (accessed: 03.09.2023).
- [25] ESA. *Venus Express*. URL: https://www.esa.int/Enabling_Support/Operations/Venus_Express. (accessed: 26.08.2023).
- [26] NAIF. *VEX Frame Kernels*. URL: https://naif.jpl.nasa.gov/pub/naif/VEX/kernels/fk/VEX_V10.TF. (accessed: 26.08.2023).
- [27] Navigation and Ancillary Information Facility. *An Overview of Reference Frames and Coordinate Systems in the SPICE Context*. URL: https://naif.jpl.nasa.gov/pub/naif/toolkit_docs/Tutorials/pdf/individual_docs/17_frames_and_coordinate_systems.pdf. (accessed: 17.05.2023).

7 Appendix

7.1 Quaternion Properties

Quaternion representation:

$$\begin{aligned}
 q &= (q_0, q_1, q_2, q_3) \\
 &= |q|(\cos(\theta/2), \sin(\theta/2)\vec{u}) \\
 &= q_0 + q_1\vec{i} + q_2\vec{j} + q_3\vec{k}
 \end{aligned} \tag{9}$$

Quaternion's vector part shares the properties with complex numbers:

$$\vec{i}^2 = \vec{j}^2 = \vec{k}^2 = \vec{i}\vec{j}\vec{k} = -1 \tag{10}$$

Quaternion's vector part shares the properties with direct rotation:

$$\begin{aligned}
 \vec{i} \wedge \vec{j} &= \vec{k} \\
 \vec{j} \wedge \vec{k} &= \vec{i} \\
 \vec{k} \wedge \vec{i} &= \vec{j}
 \end{aligned} \tag{11}$$

Conjugate of a quaternion:

$$\begin{aligned}
 q = a + \vec{v} &\Rightarrow q^* = a - \vec{v} \\
 |q| = 1 &\Rightarrow q^{-1} = q^*
 \end{aligned} \tag{12}$$

It can be concluded that: if q represents a rotation by an angle of θ , then q^* represents a rotation by an angle of $-\theta$,

Quaternion addition properties: Associative and commutative:

$$(s + \vec{v}) + (t + \vec{w}) = (s + t) + (\vec{v} + \vec{w}) \tag{13}$$

Quaternion product properties: Associative, not commutative:

$$(s + \vec{v}) \otimes (t + \vec{w}) = [st - \vec{v} \cdot \vec{w}] + [s\vec{w} + t\vec{v} + \vec{v} \times \vec{w}] \tag{14}$$

$$q_1 * q_2 \neq q_2 * q_1 \tag{15}$$

7.2 Reference frames

7.2.1 Inertial frame

The inertial frame is a frame which is non-rotating with respect to stars. Its center, the Solar System Barycenter, has a constant velocity with negligible acceleration. In this report, this frame is referred to as ICRF, or EME2000 (EME: Earth Mean Equator), or J2000. This frame is defined by:

- Reference plane: Plane of Mean Earth Equator at J2000.0
- X-axis: vernal point at J2000.0

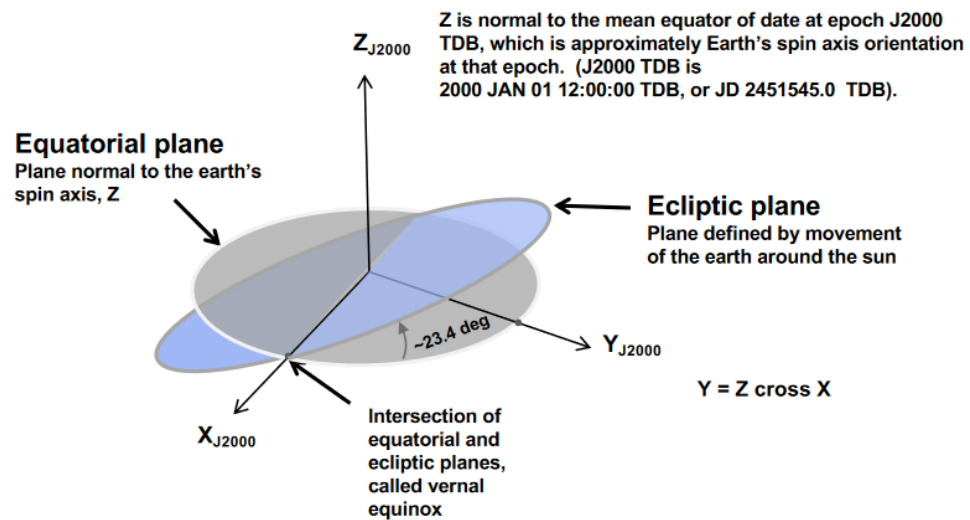


Figure 29: ICRF and Ecliptic frame, image credit: [27]

The inertial frame is one of the most frequently referred frames in every space mission. Another popular plane is the ecliptic plane, but it is not as commonly used as ICRF. The ecliptic plane is shown also in Figure 29. The reference plane of the ecliptic plane is defined by the movement of the Earth around the Sun, and this plane is 23.4° to the EME plane. The two frames share the X-axis. The Z axis of this frame is normal to the reference frame.

7.2.2 Non-inertial frames

Contrary to an inertial frame, this kind of frame has acceleration, including acceleration by rotation. Some examples of non-inertial frames:

- Body-fixed: the frame that is fixed with a body (e.g. planets, natural satellites,

etc.)hence it rotates with the body.

- Topocentric: associated with an object on or near a surface of a natural body (e.g. rover)
- Spacecraft: associated with the spacecraft's bus. In this report, it is often referred to as a "satellite frame".
- Instrument frame: The frame associated with the instrument or payloads of the satellite. Depending on how complicated the instrument is, one instrument can have one or multiple frames attached to it. This frame is also applicable to antennae, solar arrays, etc.
- Dynamic frame: Some parts of the satellites are not fixed to the bus but can change its attitude (e.g. solar array, scanning mirror). In this very early development of DOCKS' Quaternions, this family of frames is not considered yet.

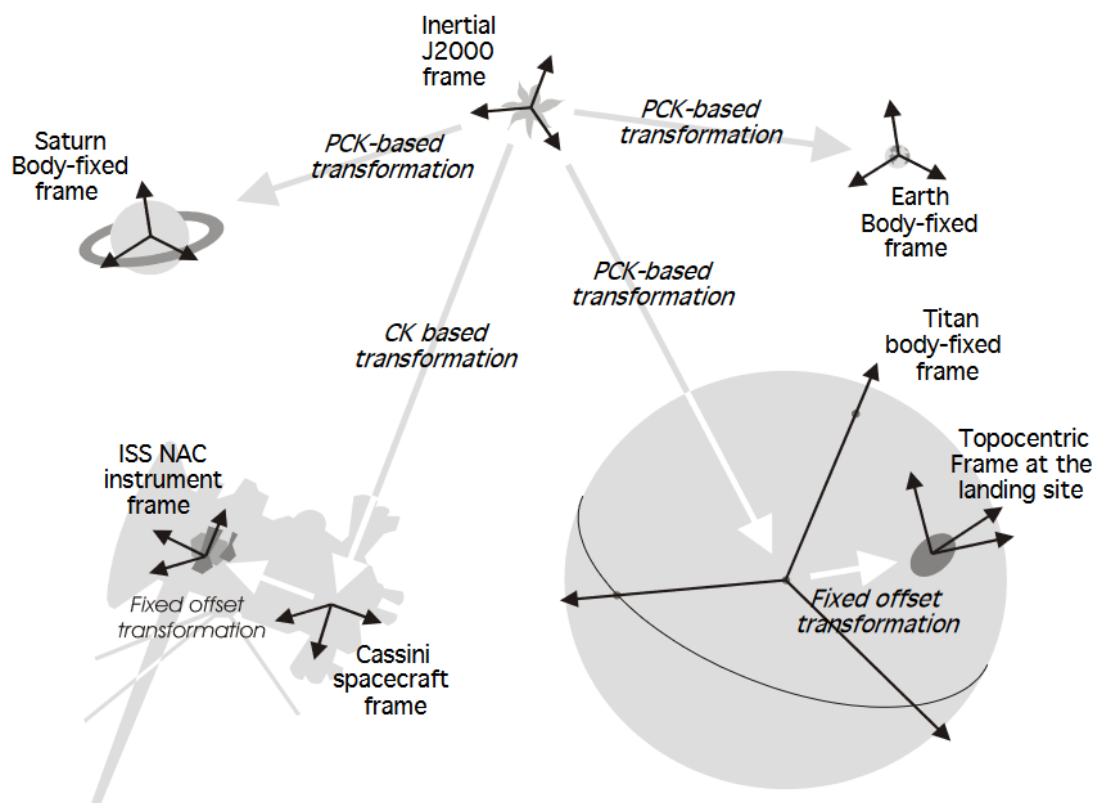


Figure 30: Family of frames, image credit: [21]

7.3 Time scale

The time scale is “a specification for measuring time: either the rate at which time passes; or points in time; or both” [SOFA Time scale, [astropy.time](#)]. A list of main time scales can be found in Table 9.

Scale	Abbreviations	Description
International Atomic Time	TAI	the official timekeeping standard
Coordinated Universal Time	UTC	the basic of civil time
Universal Time	UT1	based on Earth rotation
Terrestrial Time	TT	used for Solar system ephemeris look-up
Geocentric Coordinate Time	TCG	used for calculations centered on Earth in space
Barycentric Coordinate Time	TCB	used for calculations beyond Earth orbit
Barycentric Dynamical Time	TDB	a scaled of TCB that keeps in step with TT on the average.

Table 9: Time scales

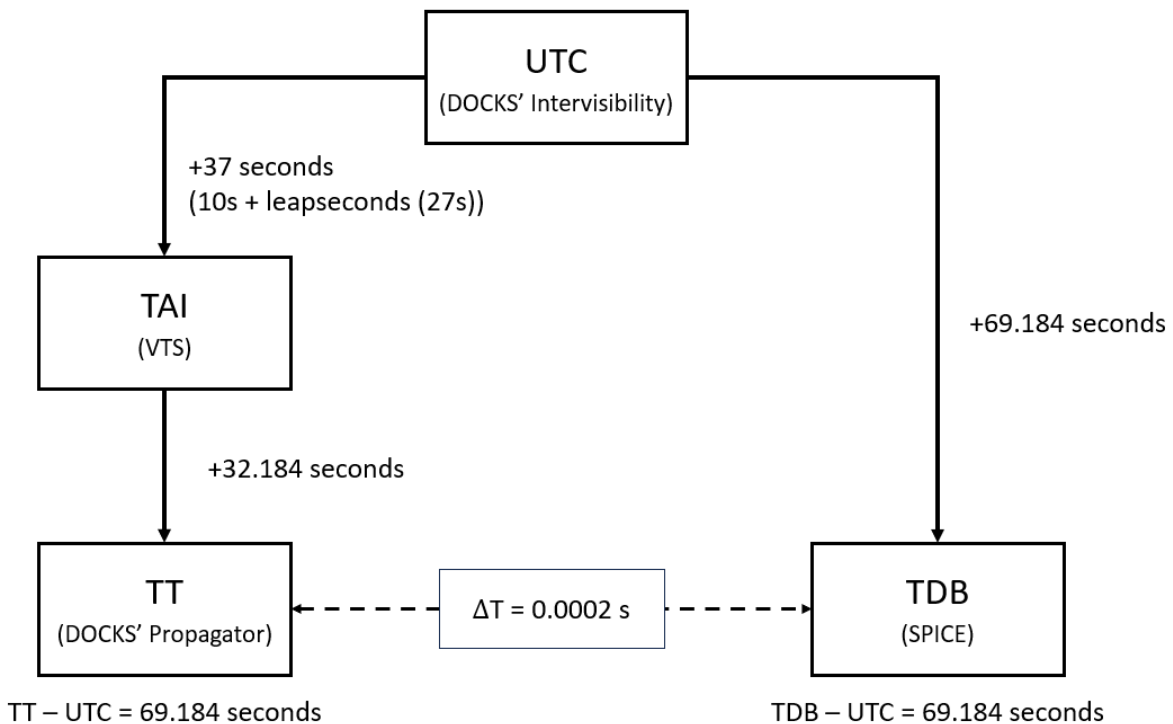


Figure 31: Time scale relations, image adapted from F. Lacreu (CENSUS)

In Figure 31, it is noticed that each platform and data set has a preferred time scale, and it is important to know the time scale that the ephemeris presented in. [astropy.time](#) is a powerful package to deal with the timescale transformation. However, since the

information regarding leapsecond is unpredictable, *astropy.time* cannot assure the accuracy for time conversion in the forthcoming time. NAIF's SPICE also contains specific kernels for past leapseconds, and this kernel is required whenever solar system bodies' ephemeris is utilized.



UNIVERSITE DES SCIENCES ET DES TECHNOLOGIES DE HANOI
UNIVERSITY OF SCIENCE & TECHNOLOGY OF HANOI
TRƯỜNG ĐẠI HỌC KHOA HỌC & CÔNG NGHỆ HÀ NỘI

SUPERVISOR'S APPROVAL FOR INTERNSHIP THESIS CORRECTION

Topic : DÉVELOPPEMENT D'UN MODULE D'ORIENTATION DE
NANOSATELLITE POUR LA SUITE LOGICIEL DOCKS POUR LE
PROTOTYPAGE PRÉCOCE DE PROFILS DE MISSIONS SPATIALES

Specialty : Space Technology

Student's full name : La Thuy Linh

Supervisor's full name : Feliu Lacreu

Host organization : Observatoire de Paris

I hereby agree with all the Internship thesis corrections based on the Jury comments from student La Thuy Linh. This report is qualified to be a Master thesis and to be archived in the USTH library for any references.

Hanoi, 18 /09/2023

Reviewer

(Signature and full name)

Linda Tomasini

Supervisor

(Signature and full name)

Feliu Lacreu